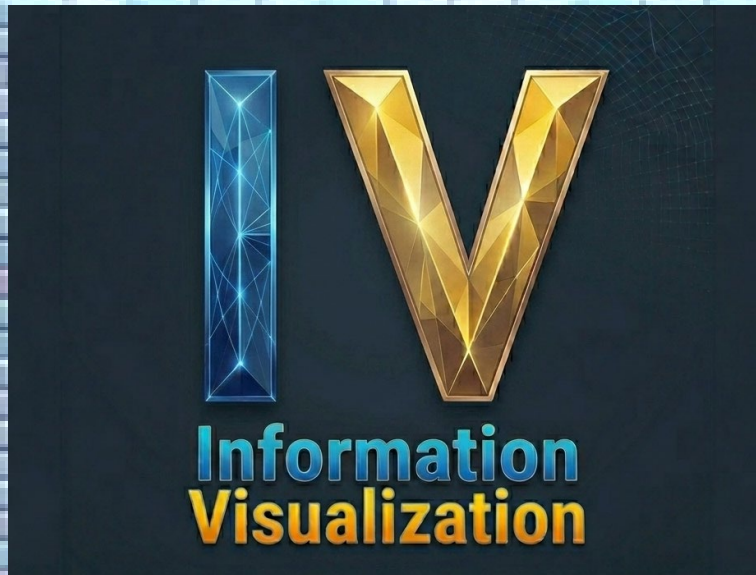


Vorlesung
Informationsvisualisierung



Kapitel 8

SOFTWAREVISUALISIERUNG



Lecture
Information Visualization

Chapter 8

SOFTWARE VISUALIZATION

Prof. Dr.
Stephan Diehl

Universität Trier – Sommersemester 2026

Next Tutorial

2

- Tuesday, **30.06.2026**, 10-12 in HZ 203
 - 7th and 8th exercise sheet

Preliminary Schedule

3

Thursday, 12-14		
16.04.2026	Introduction	
23.04.2026	Infographics	
30.04.2026	Visual perception and cognition	
07.05.2026	Visualization of hierarchical data	
14.05.2026	Christi Himmelfahrt	
21.05.2026	Visualization of graphs	
28.05.2026	Visualization of multivariate data and time series	
04.06.2026	Fronleichnam	
11.06.2026	Interaction	
18.06.2026	Software visualization	Software visualization
25.06.2026	Software visualization	Software visualization
02.07.2026	Some empirical results	Software visualization
09.07.2026	Exam preparation	Some empirical results & Exam preparation
16.07.2026	Final exam	

Chapter
8.1

Software Visualization

Brainstorming



5

*Examples of
Software Artifacts?*

Software Artifacts

6

Technical Artifacts

- Source code
- Data structures
- Representations in memory (at runtime)

Other Artifacts

- Requirements
- Software design
- Bug reports
- Code changes

Definition: Software Visualization

7

Visualization of artifacts related to software and its development process

... in other words ...

*Visualizuation of the **structure, behavior and evolution** of software*

Software Visualization

8

Structure

- Code and Data structures
- Software architecture
- Dependencies between software artifacts

Behavior

- Execution of an algorithm
- Runtime behavior
- Program state

Evolution

- Development history of a software system
- (Sequences of) source code changes
- Team building and development

Chapter
8.2

Structure: Code Visualization

10

Code Layout

Pretty Printing & Syntax Highlighting

11

- Program text = sequence of characters
- Pretty printing
 - ▣ Line breaks to discern statements
 - ▣ Indentation to make the structure more explicit

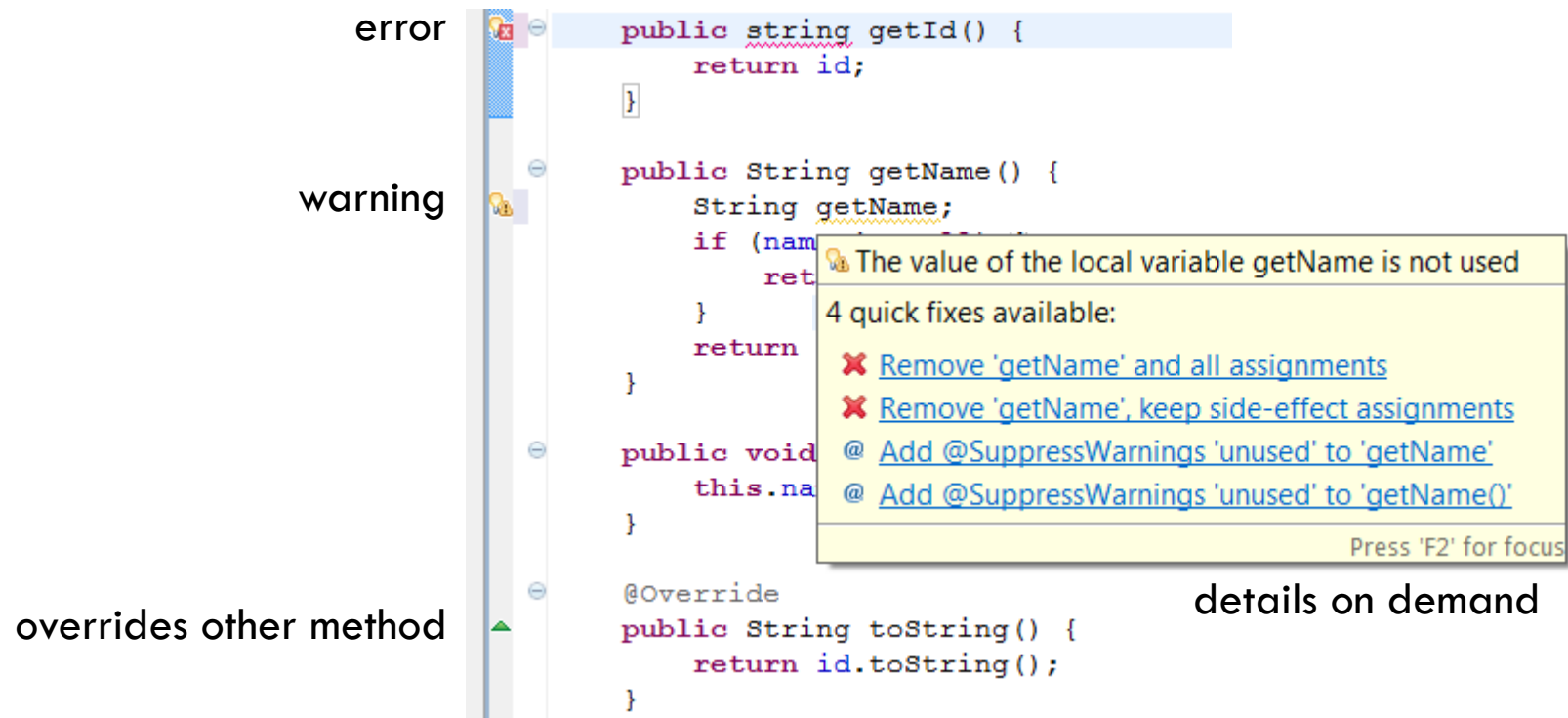
```
int i,c; while(i<100) if (i%2==0) c++; i++;
```

- Syntax highlighting
 - ▣ Font face and colors to highlight the syntax

```
int i, c;  
  
while (i<100)  
    if (i%2 == 0)  
        c++;  
        i++;
```

Augmenting source code with additional information

12



➔ parse and compile the code in the background

SeeSoft: Visualizing thousands or millions of lines of code

13

- Color Coding of Software Metrics
- Textual Representation
- Line Representation
 - Each line of text is represented by a colored line of pixels
- Pixel Representation
 - Each line of text is represented by a one (or a few) pixel.
 - Order of pixels:
 - According to order of the lines in the text
 - According to color

proportional

Linie Representation

14

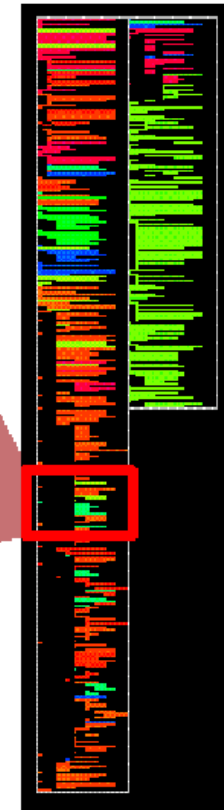
```

if (skip)
{
    fprintf(prn_stream, "\\033J%c", skip);
}

/* Copy the scan lines */
lcnt = gdev_rpn_copy_scan_lines(pdev, in,
if (lcnt < 8 * in_y_mult )
{ /* Pad with lines of zeros */
    memset(in + lcnt*line_size, 0,
           in_size - lcnt * line_size);
}

if (y_spin_high)
{ /* Shuffle the scan lines */
    byte *p;
    int i;
    static const char index[] =
    { 0, 8, 16, 1, 9, 17,
      2, 10, 18, 3, 11, 19,
      4, 12, 20, 5, 13, 31,
      6, 14, 22, 7, 15, 23
    };
    for(i = 0; i < 24; i++)

```

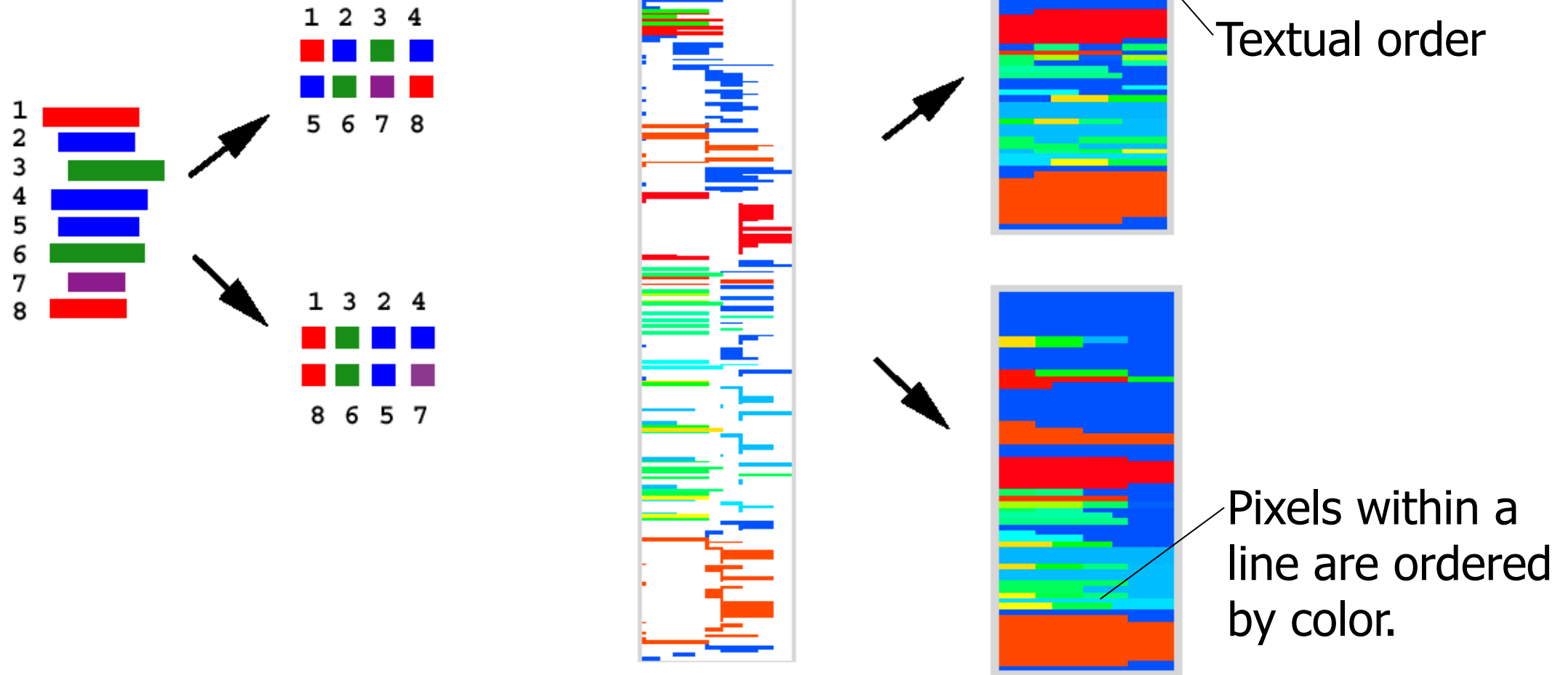
[illegible]

Color can encode different software metrics:

- Size of moduls
- Runtime of program
- Number of Changes
- Number of Bugfixes
- Number of
programmers that did
a change
- Depth of nested blocks
- Type of error
- ...

Pixel Representation

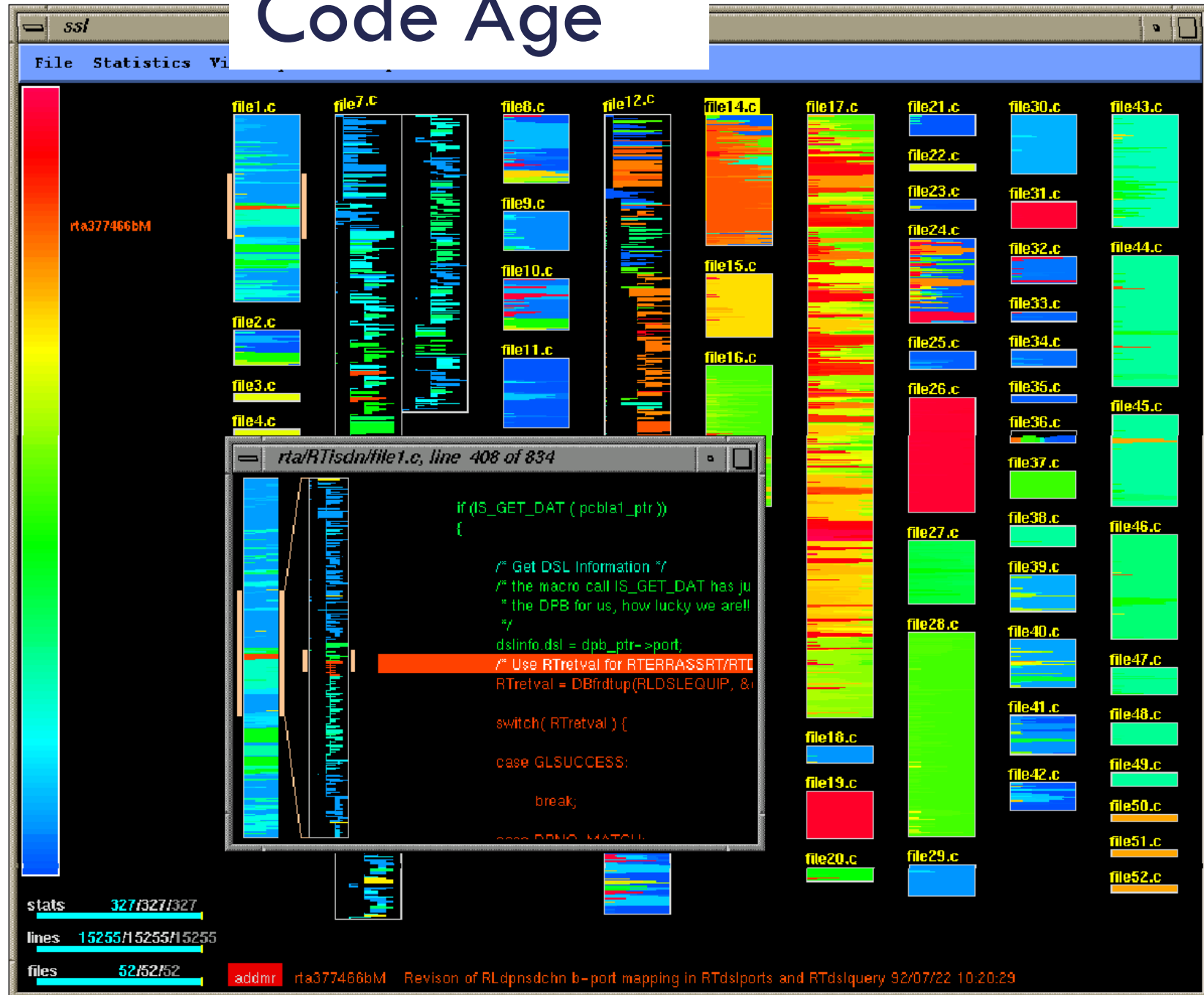
15

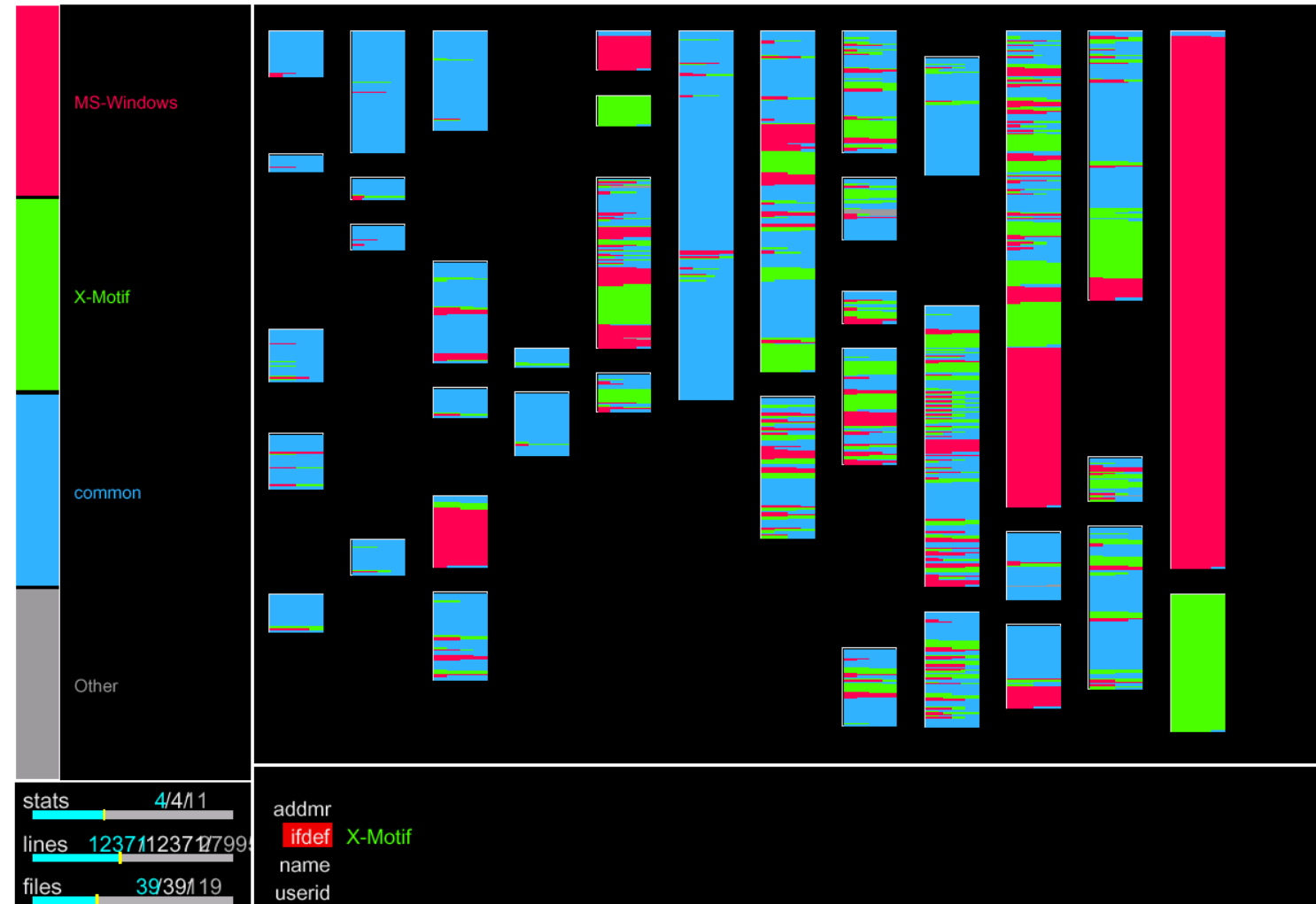
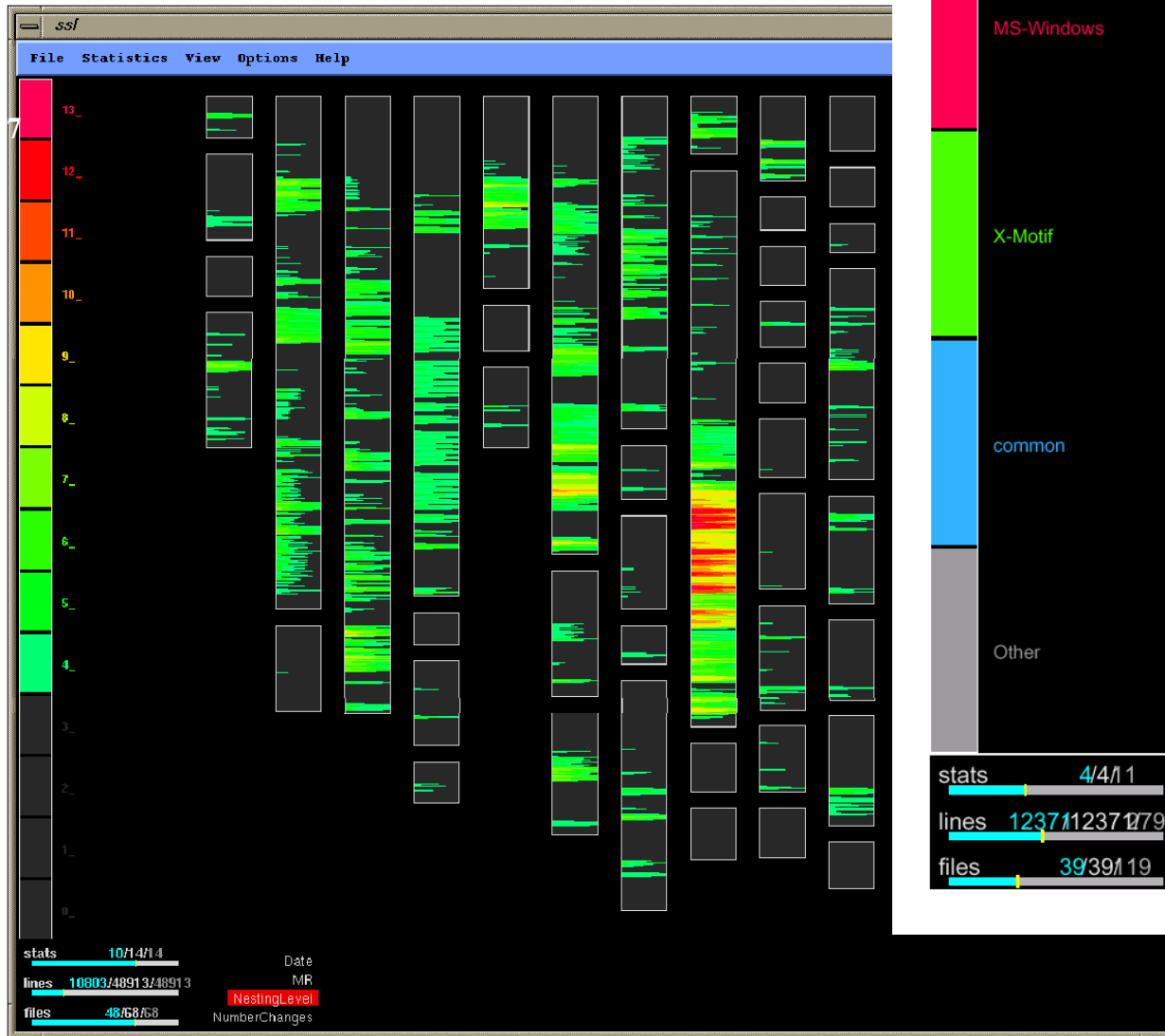


Code Age

16

Color indicates time of most recent change of line of code.





Version-specific Code

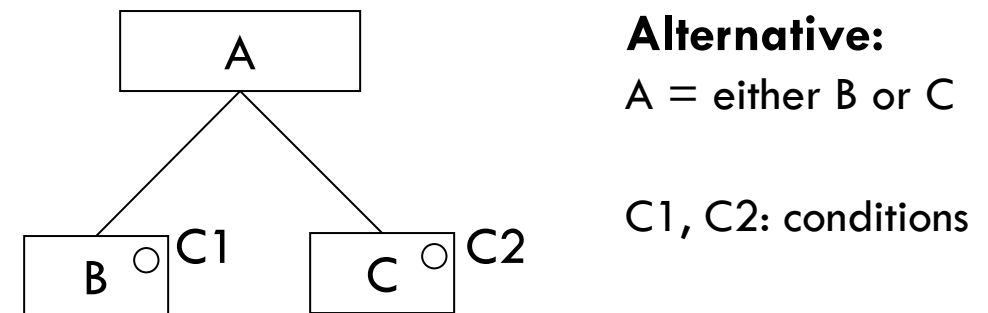
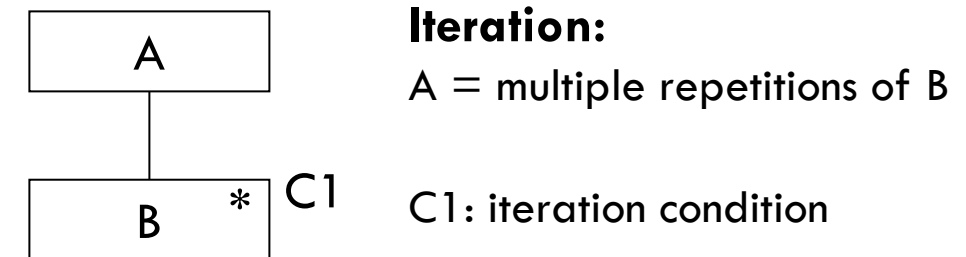
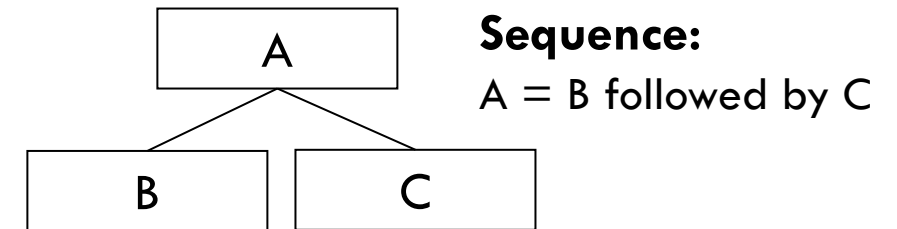
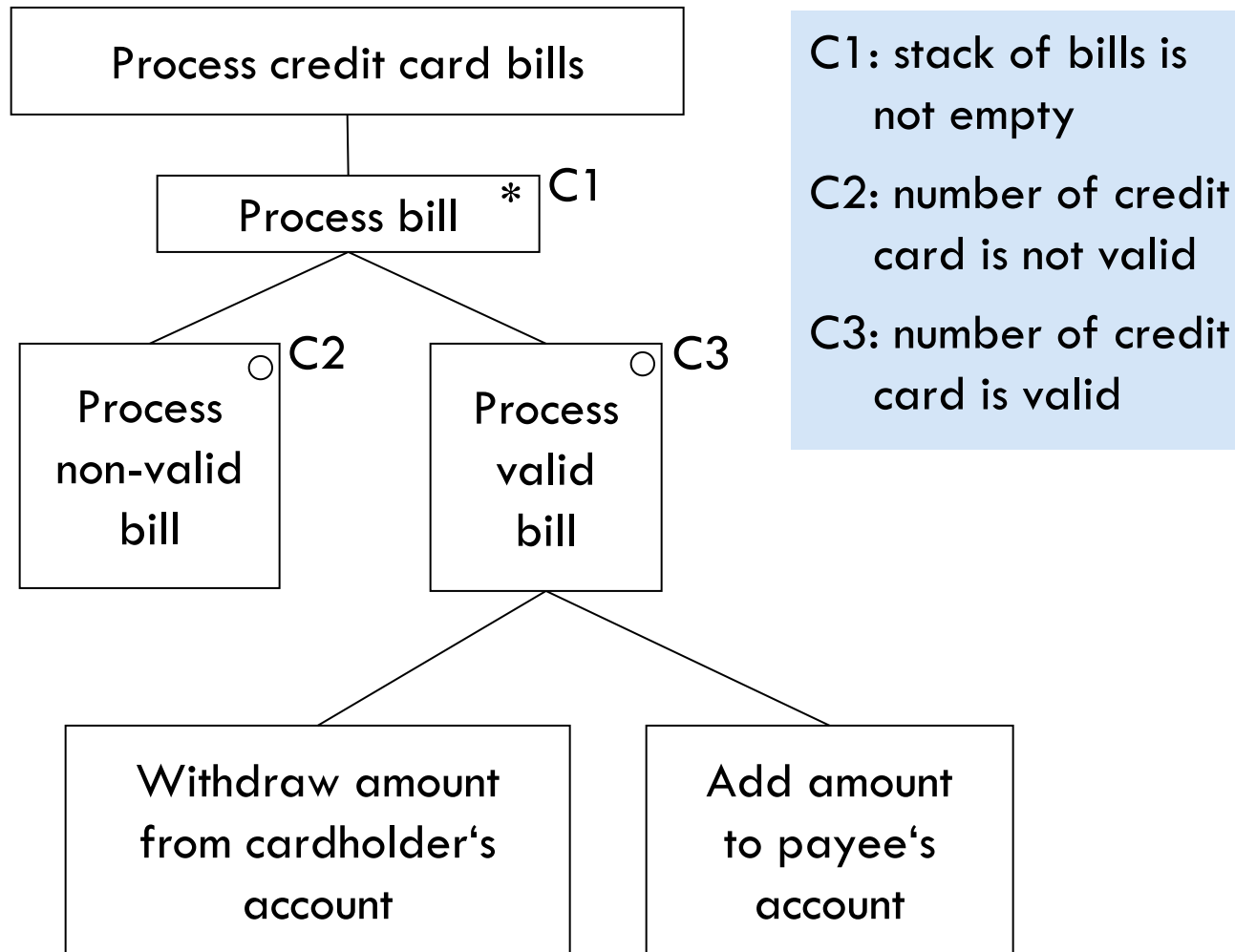
Depth of nested blocks

18

Code Diagrams

Jackson Diagrams

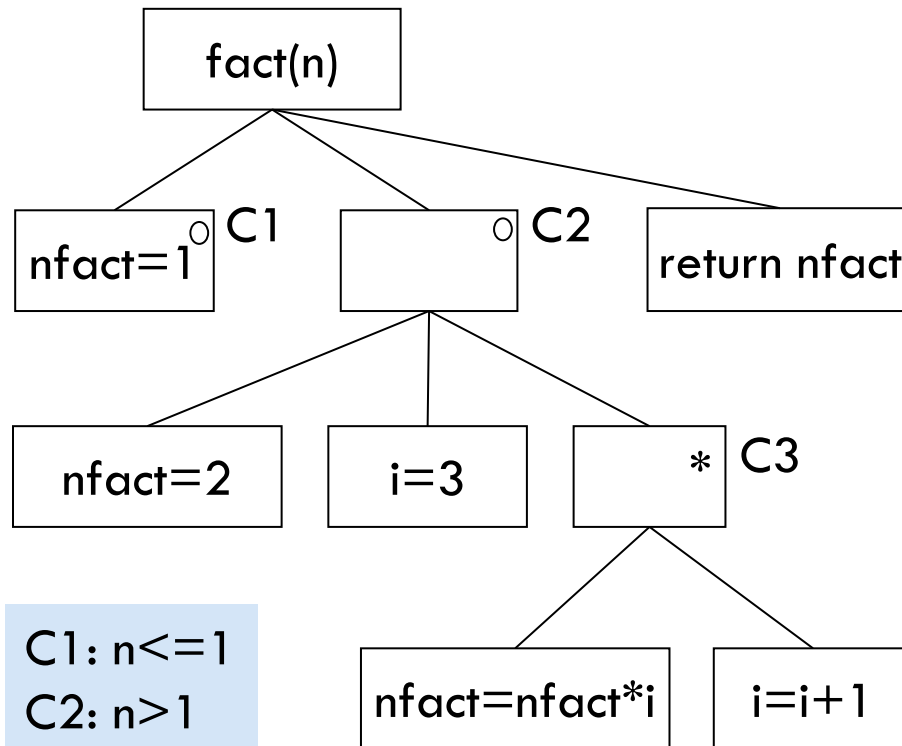
19



Jackson Diagrams: Example

20

➔ more usable for design than for visualizing actual code



C1: $n \leq 1$
C2: $n > 1$
C3: $i \leq n$

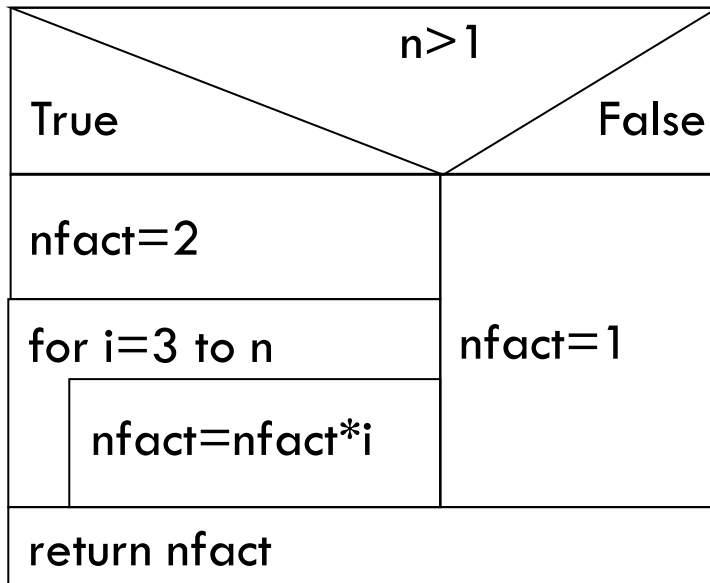
```
int fact(n)
{
    if (n>1)
    {
        nfact=2;
        for(int i=3;i<=n;i++)
            nfact=nfact*i;
    }
    else
    {
        nfact=1;
    }
    return nfact;
}
```

Nassi-Shneiderman Diagrams

also known as “Structograms”
use nested boxes to visualize structure

21

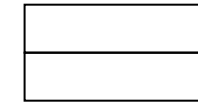
fact(n)



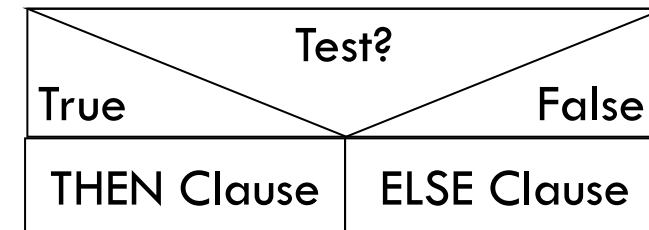
```
int fact(n) {
    if (n>1) {
        nfact=2;
        for(int i=3; i<=n; i++)
            nfact=nfact*i;
    } else { nfact=1; }
    return nfact;
}
```



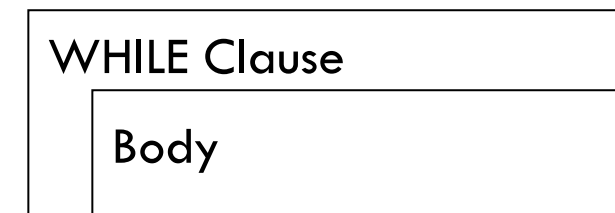
Process



Sequence



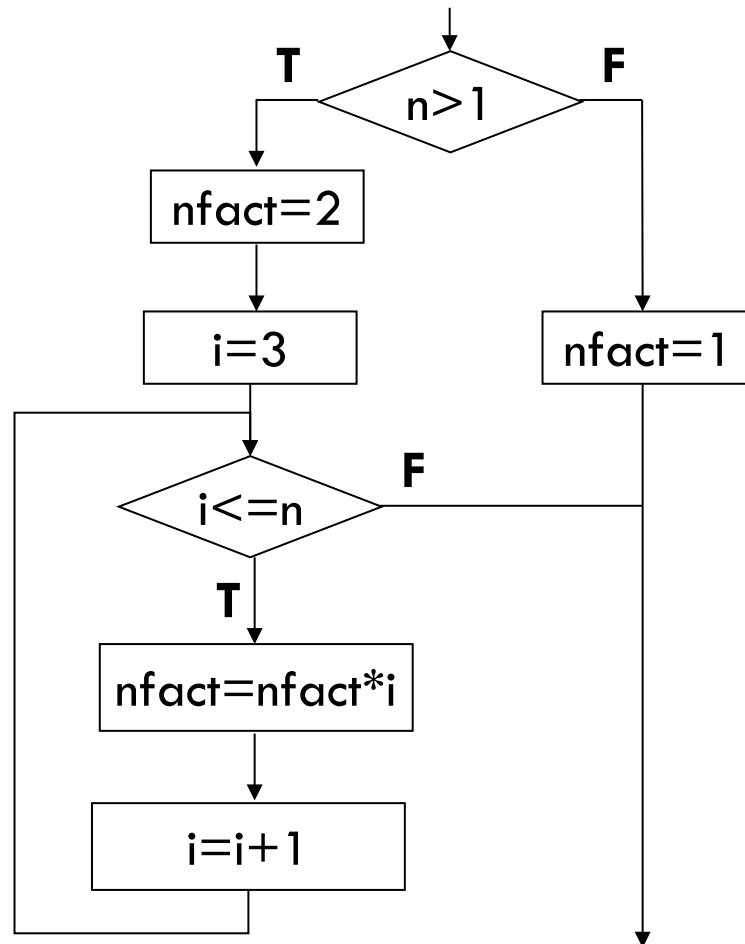
Conditional



Loop

Control-Flow Graph: Example

22



```

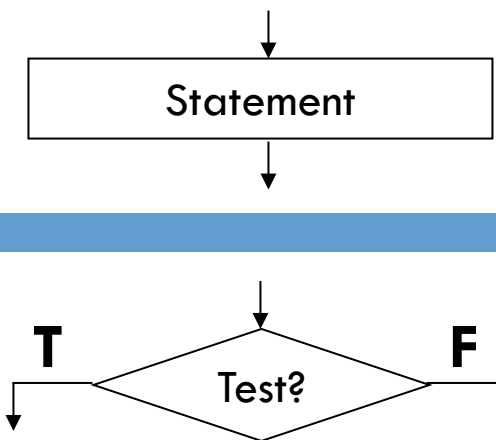
int i=3;
while(i<=n) {
    nfact=nfact*i;
    i=i+1;
}
  
```

```

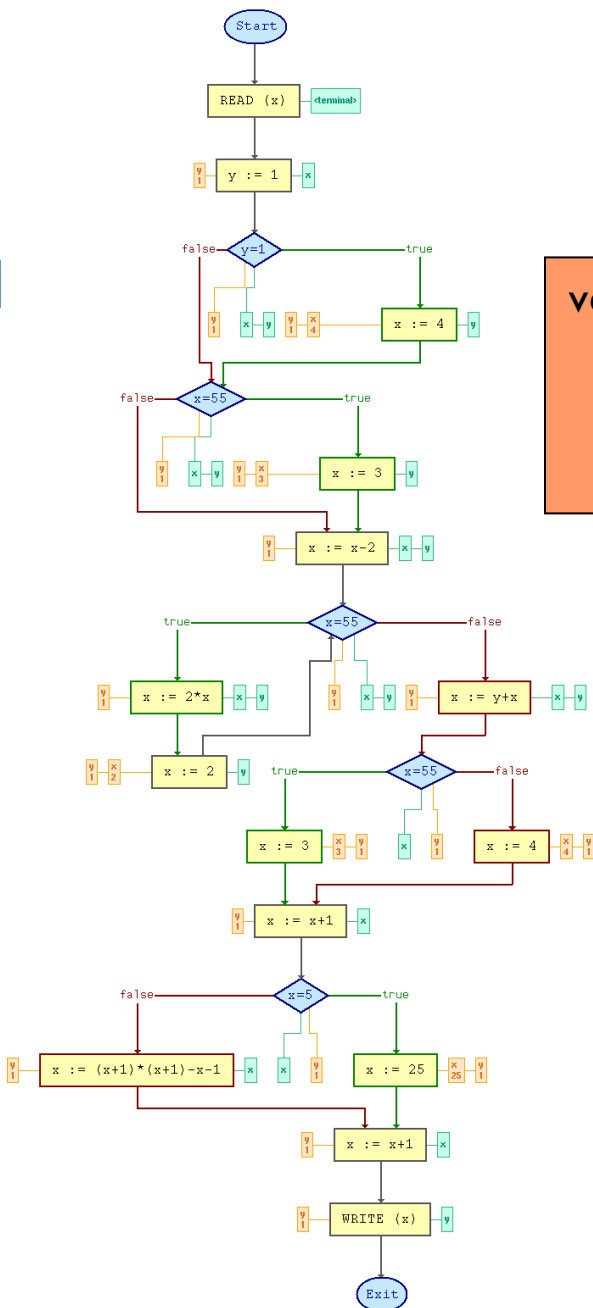
int fact(n) {
    if (n>1) {
        nfact=2;
        for(int i=3; i<=n; i++)
            nfact=nfact*i;
    } else { nfact=1; }
    return nfact;
}
  
```

Arrow indicates temporal order or flow of control.

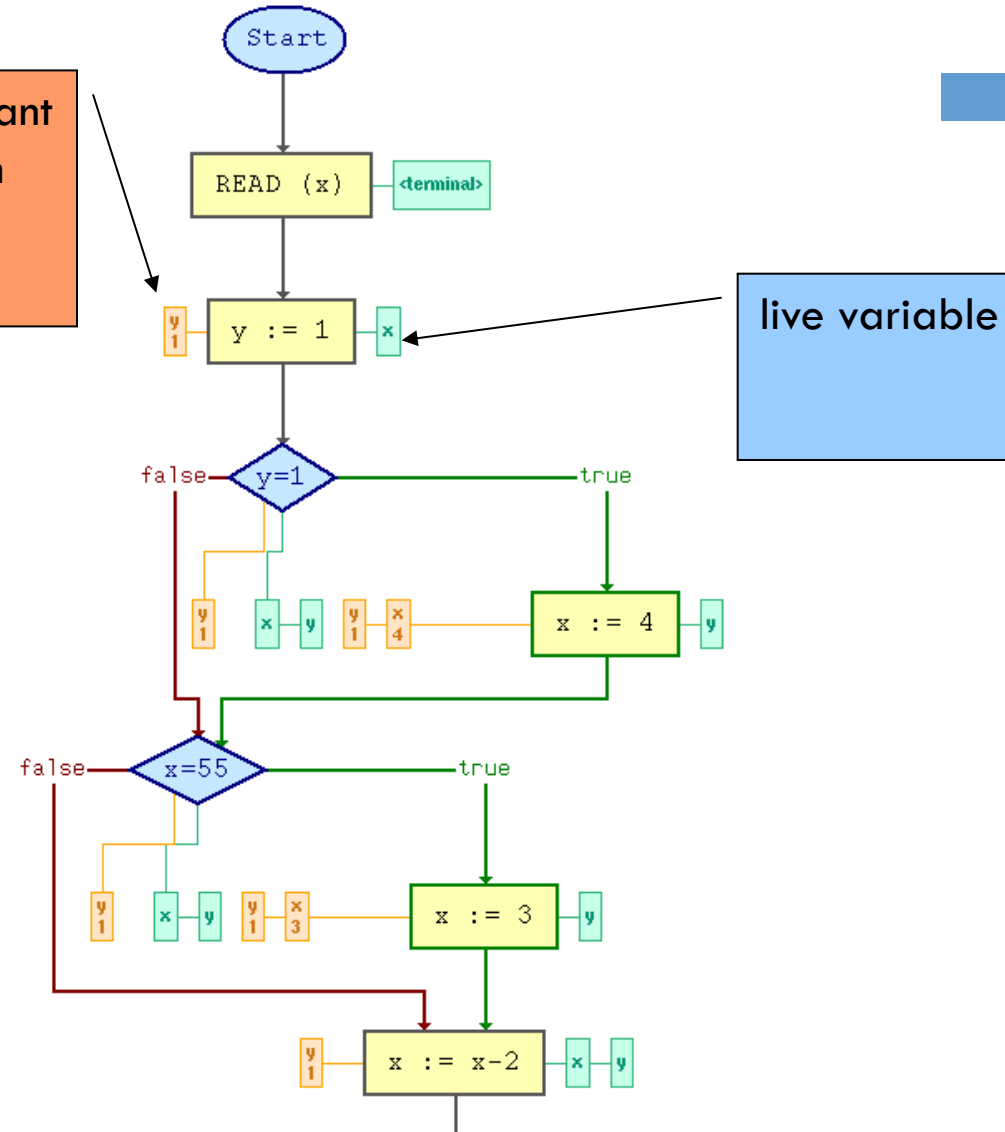
Flow of control means what statement is executed next by the control (e.g. CPU, execution unit)



Live Variables with aiSee



value for constant propagation



Extra slides – not covered in lecture !

Computation of Control-Flow Graphs

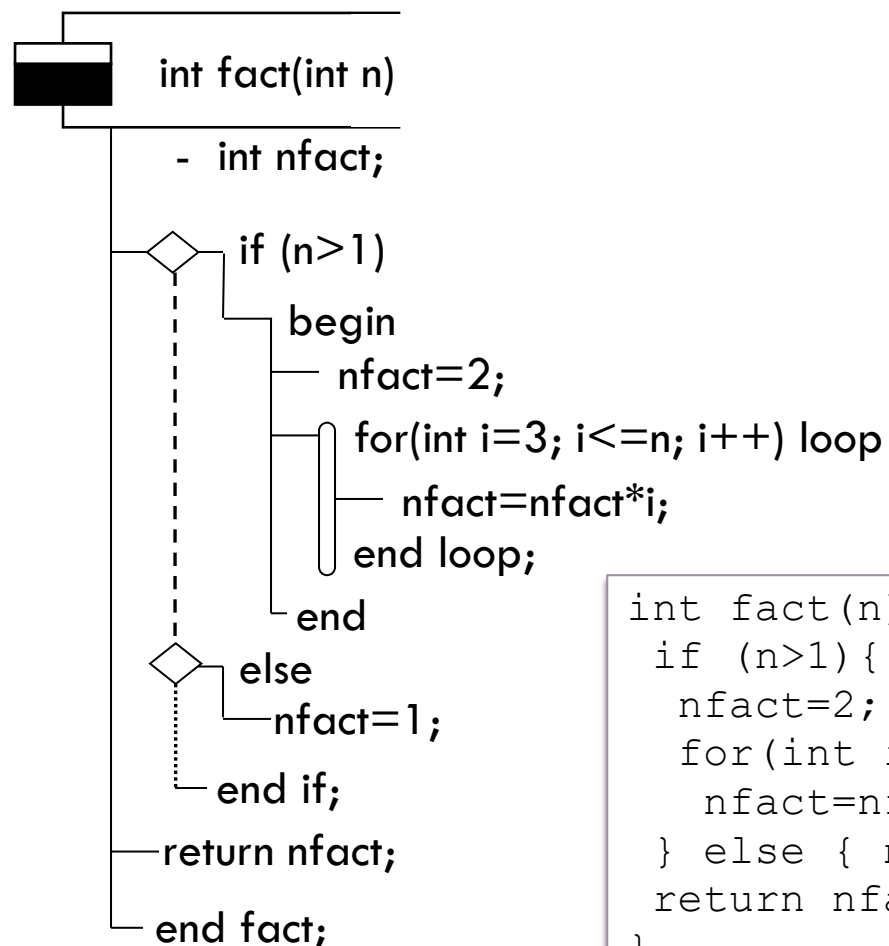
→ Modul “Übersetzung und Analyse von Programmen”

Template-based Layout of Control-Flow Graphs

The extra slides are included at the end of this presentation.

Control-Structure Diagrams

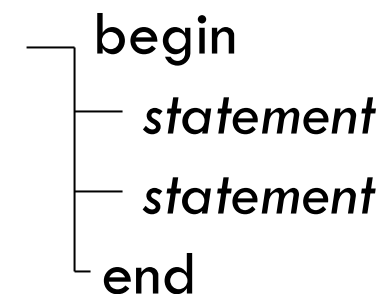
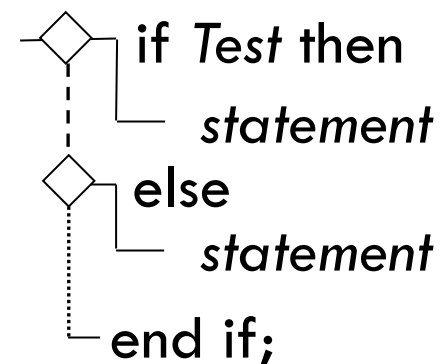
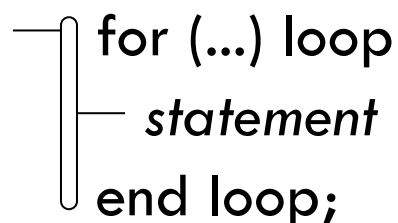
25



```

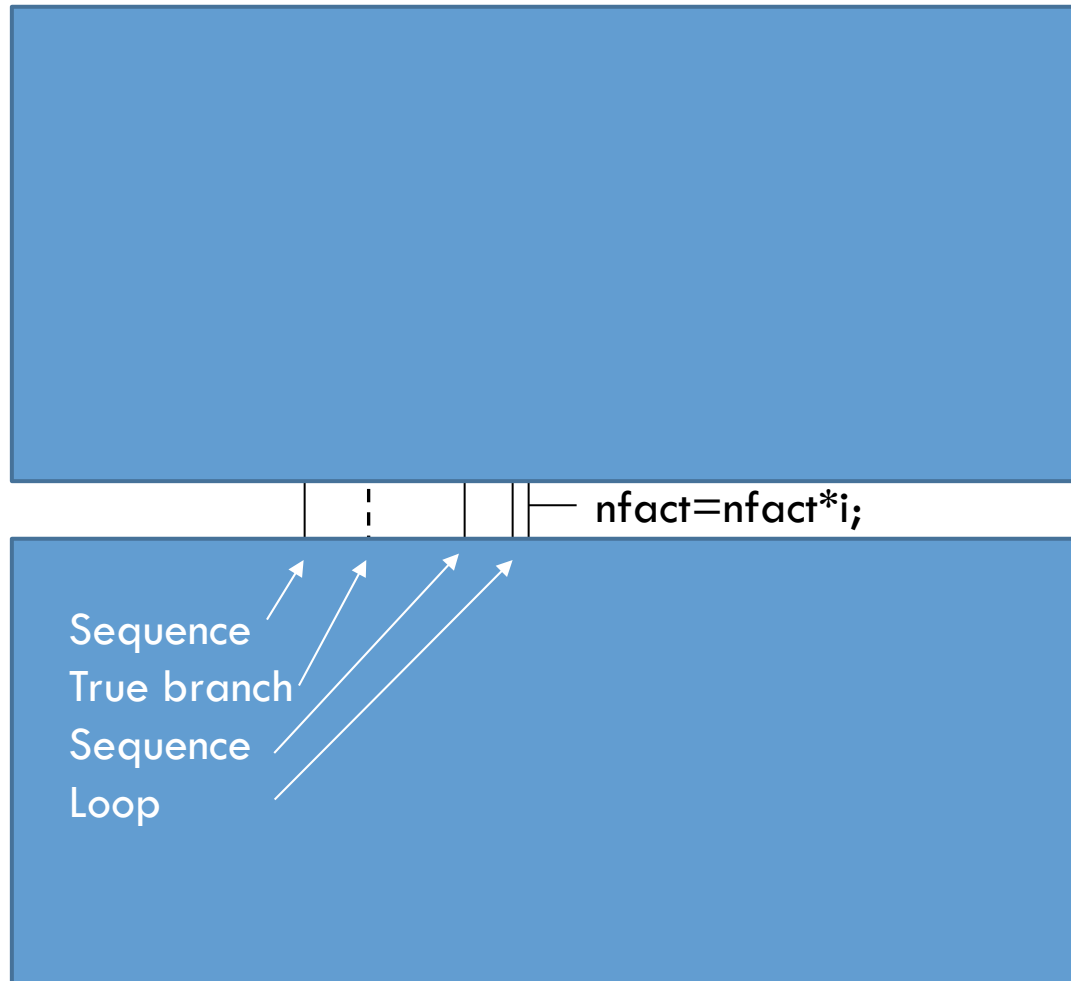
int fact(n) {
    if (n>1){
        nfact=2;
        for(int i=3; i<=n; i++)
            nfact=nfact*i;
    } else { nfact=1; }
    return nfact;
}

```

**Sequence****Conditional****Loop**

Control-Structure Diagrams: Example

26

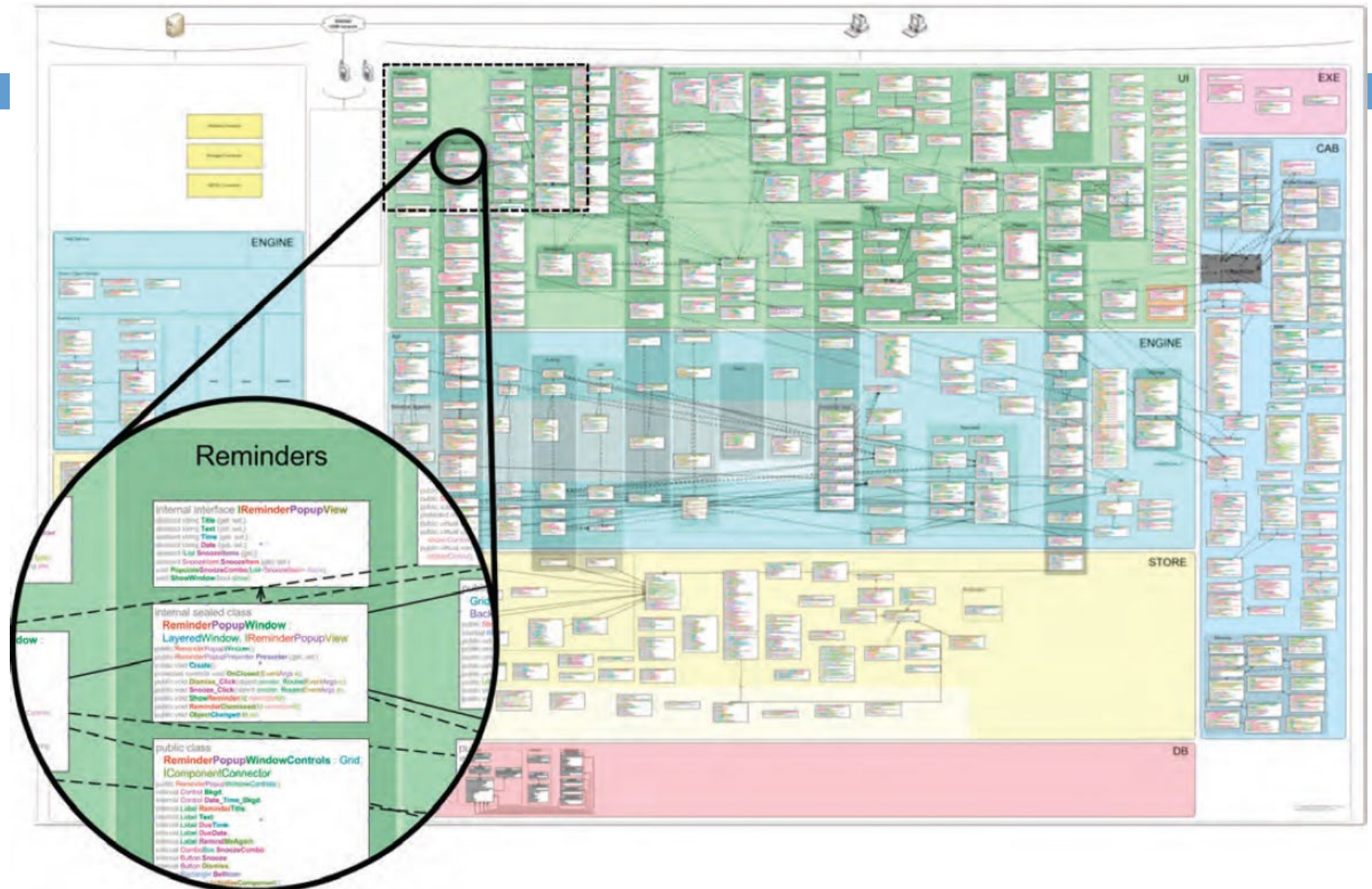


```
int fact(n)
{ if (n>1)
    { nfact=2;
      for(int i=3;i<=n;i++)
        nfact=nfact*i;
    }
  else
    { nfact=1;
    }
  return nfact;
}
```


Code Canvas

<http://research.microsoft.com/en-us/projects/codecanvas/>

28



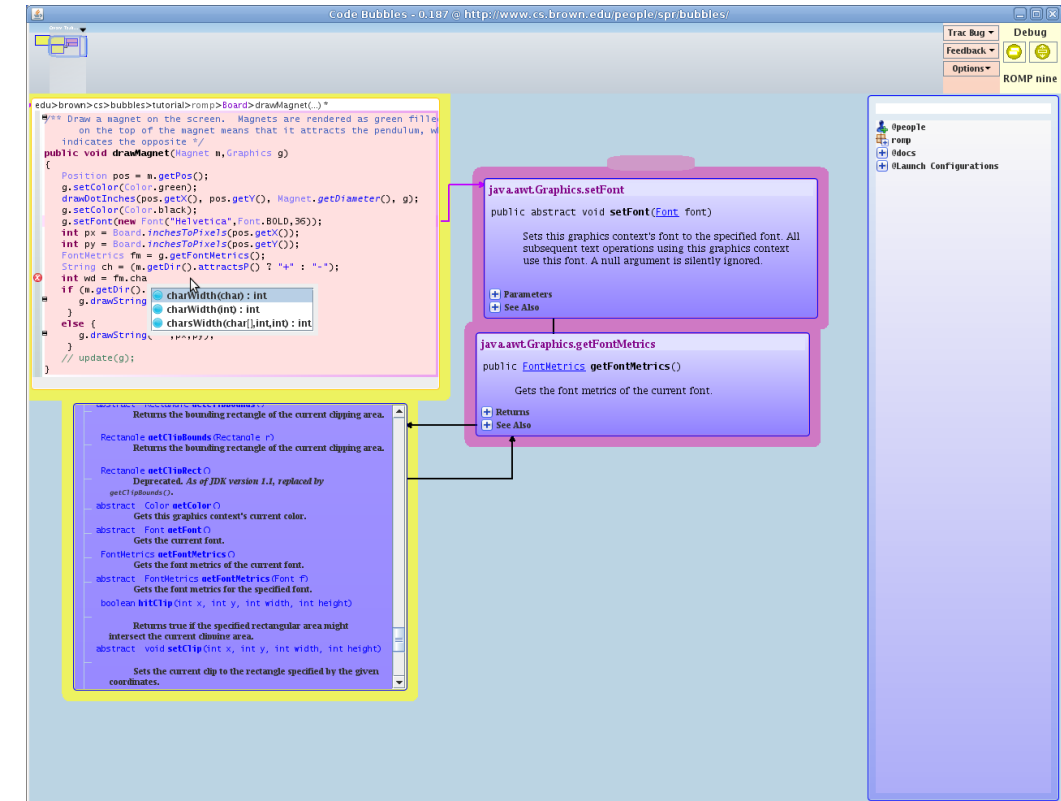
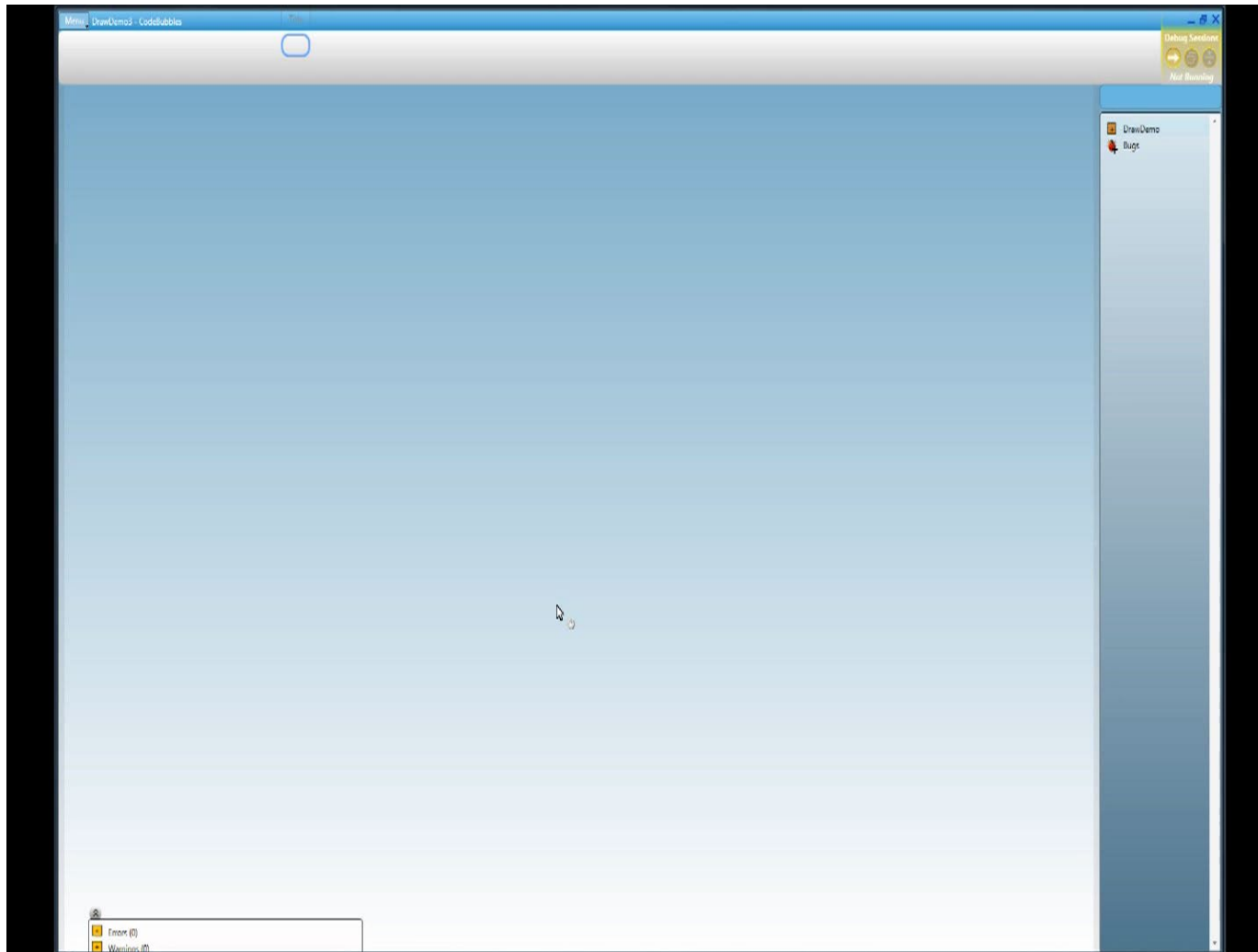
28



Code Bubbles

Source: "[Code Bubbles: Rethinking the User Interface Paradigm of Integrated Development Environments](#)", by Andrew Bragdon, Steven Reiss, Robert Zeleznik, Suman Karumuri, William Chang, Joshua Kaplan, Christopher Coleman, Ferdi Adeptura, and Joseph LaViola, Jr (ICSE 2010)

29



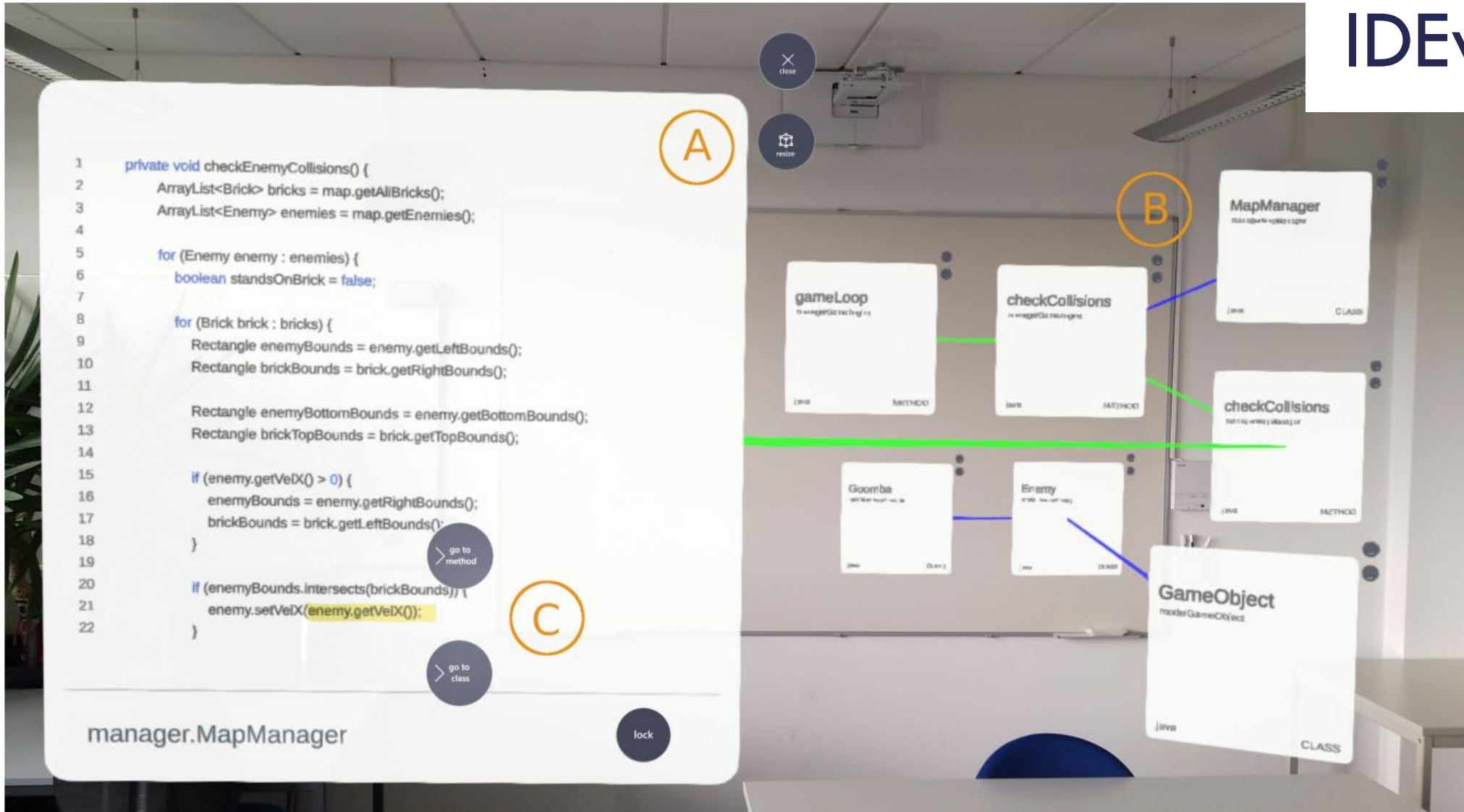


Fig. 3. IDEVELOPAR in practice. The exemplary procedure from section III is shown. (A) Opened code panel (B) Two independent, visual linked emerging code panel trees shown at a different level of detail (semantic zooming) (C) Code navigation, when clicking on a followable code element

Source: Lucas Kreber, Stephan Diehl, and Patrick Weil, 10th IEEE Working Conference on Software Visualization (VISOFT 2022), 2022.

Chapter
8.3

Architecture Visualization

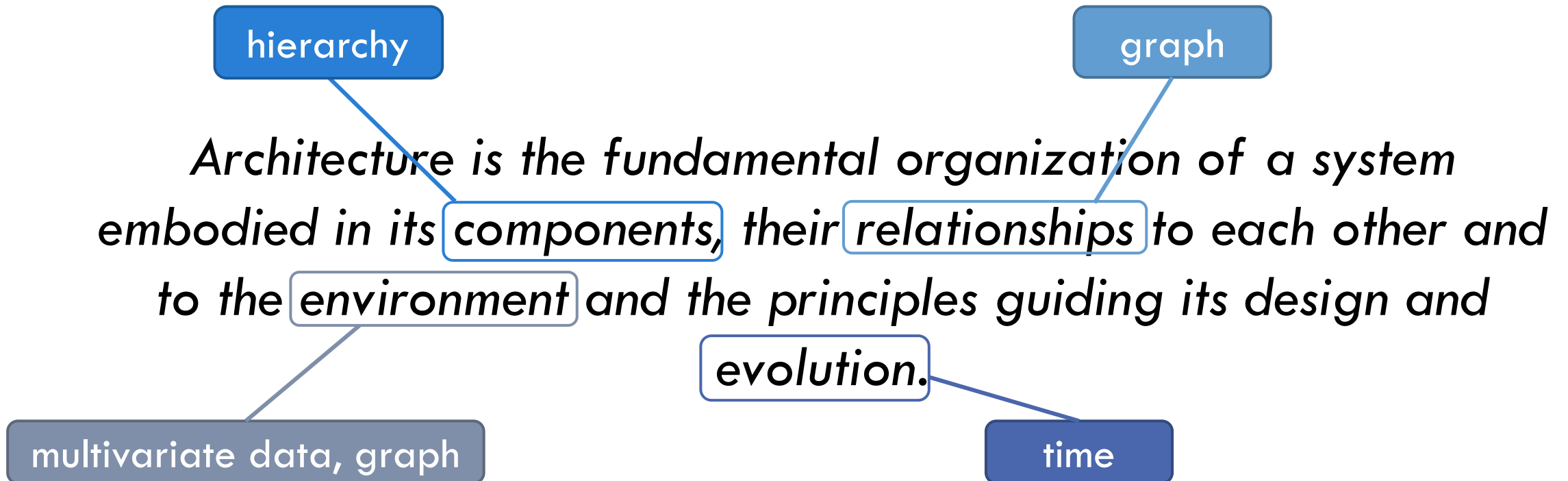


32

Structure: Software Architecture

Software Architecture

33



Source: IEEE recommended practice for architecture description of Software-Intensive Systems , IEEE Standard 1471, 2000.

Some Common Architectures

34

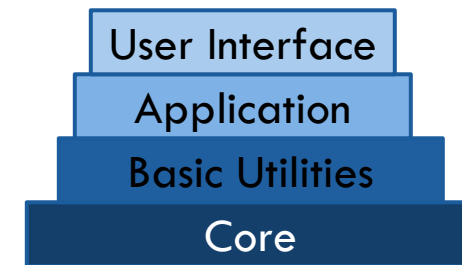
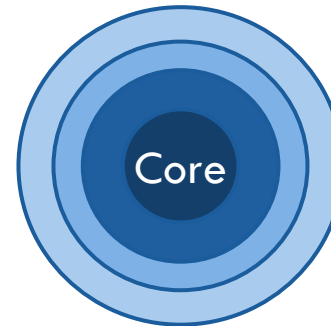
Pipes and Filters

- input stream → output stream



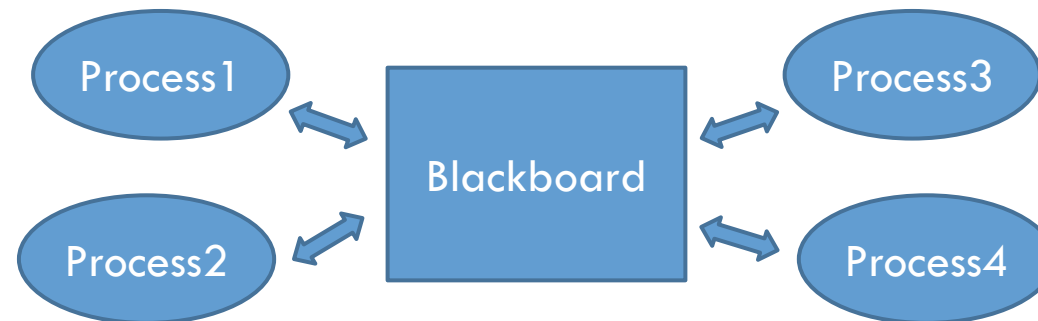
Layered Systems

- only access from higher to lower layers



Blackboard-driven

- multiple units
- read&write



Ad-Hoc Diagrams

35

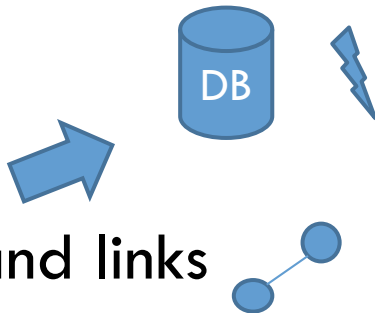
□ many architecture diagrams follow no standard notation

□ typical graphical elements

□ icons

□ arrows

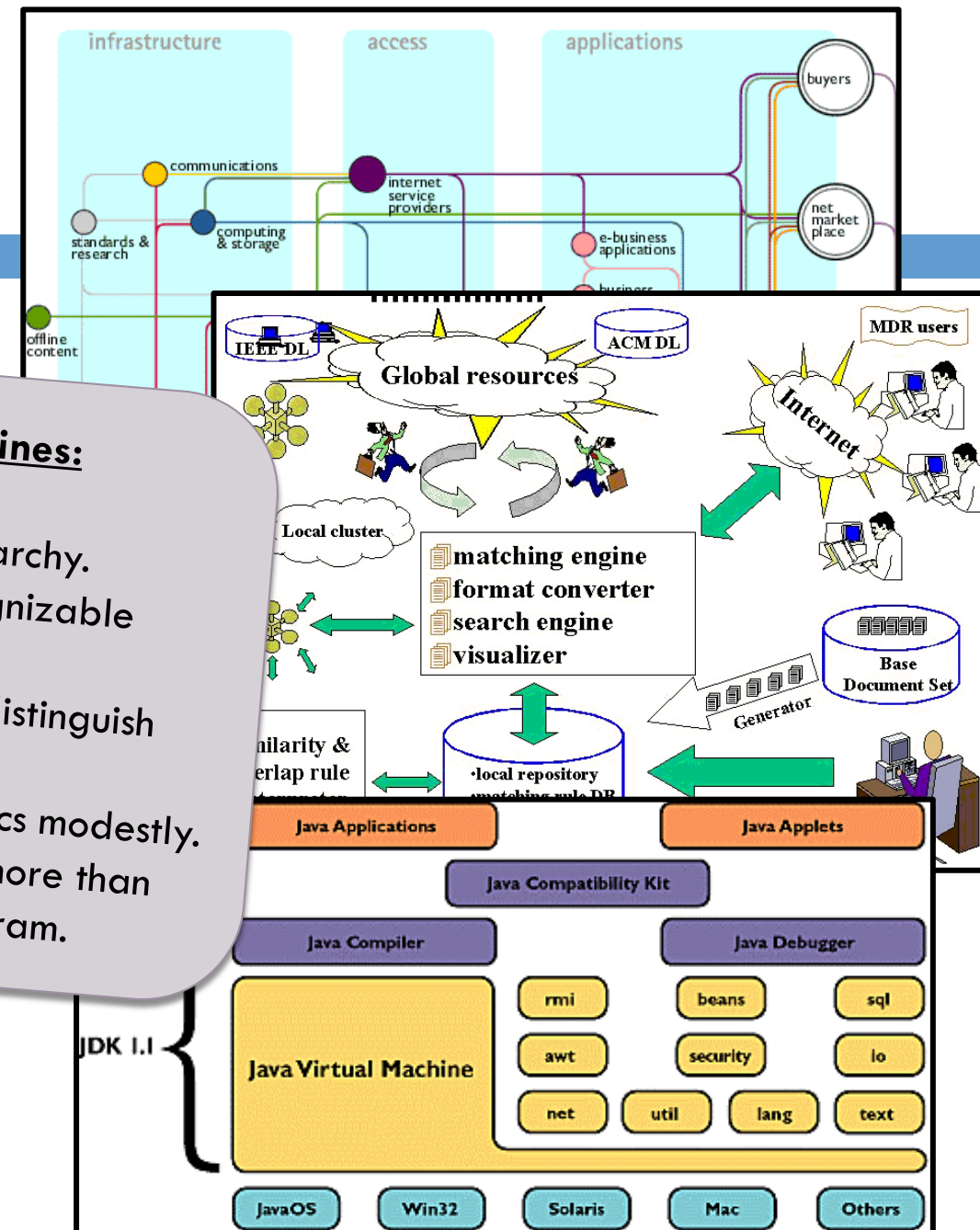
□ nodes and links



Some Guidelines:

- Create a clear hierarchy.
- Chose a clear, recognizable positioning pattern.
- Use white space to distinguish objects.
- Use icons and graphics modestly.
- Use color; don't use more than six colors in one diagram.

Source: Henk Koning, Claire Dormann, and Hans van Vliet. 2002. Practical guidelines for the readability of IT-architecture diagrams. 20th annual international conference on Computer documentation (SIGDOC), 2002. <https://doi.org/10.1145/584955.584969>



Unified Modeling Language (UML)

36

- a set of graphical notations for modeling software systems
- combination of the methods of Booch, Rumbaugh und Jacobsen („Three Amigos“).
- standardized by the Object Management Group (OMG)

use case diagram

class diagrams

behavior diagrams

- statechart diagram
- activity diagram

interaction diagrams

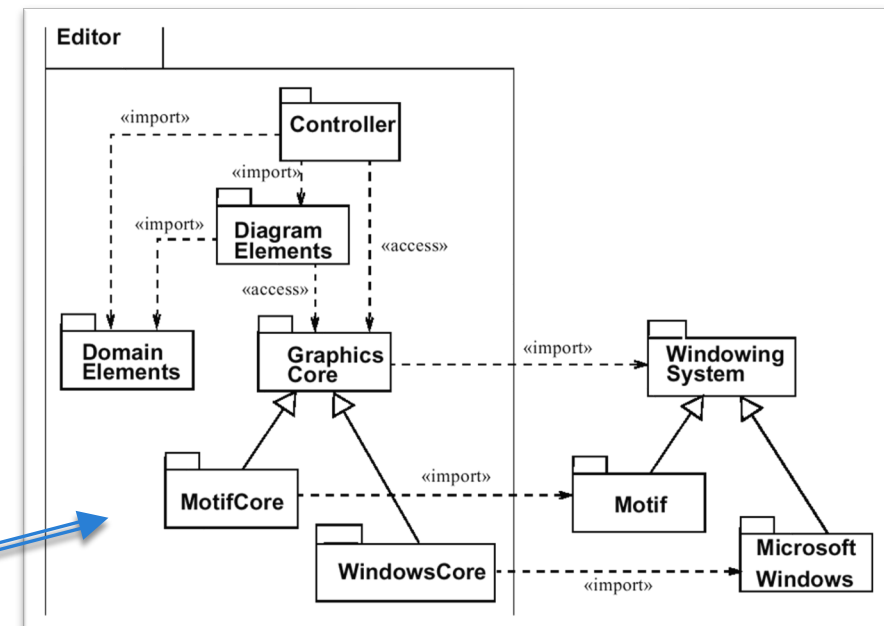
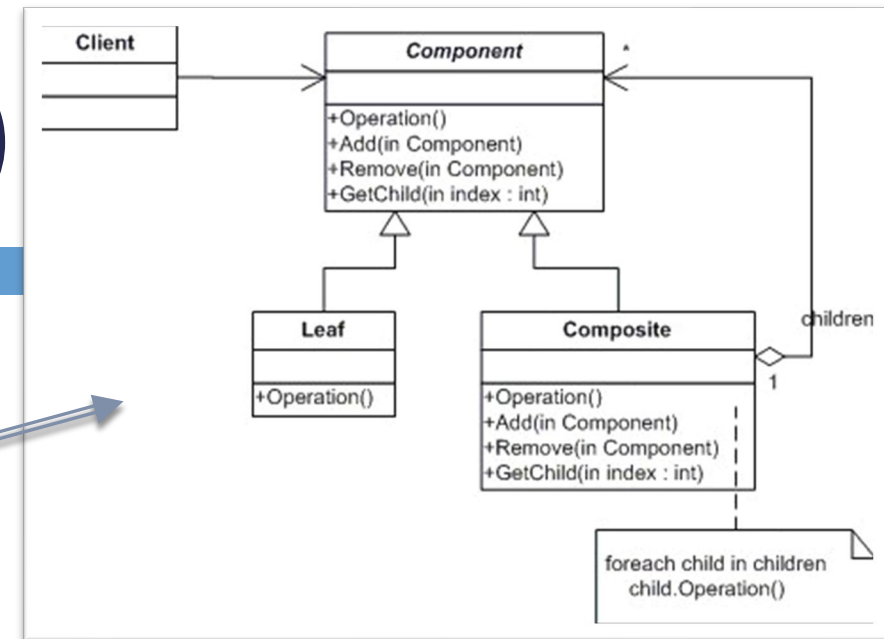
- sequence diagram
- collaboration diagram

implementation diagrams

- component diagram
- deployment diagram

model management diagrams

- subsystems, models and packages



Visually more efficient representations?

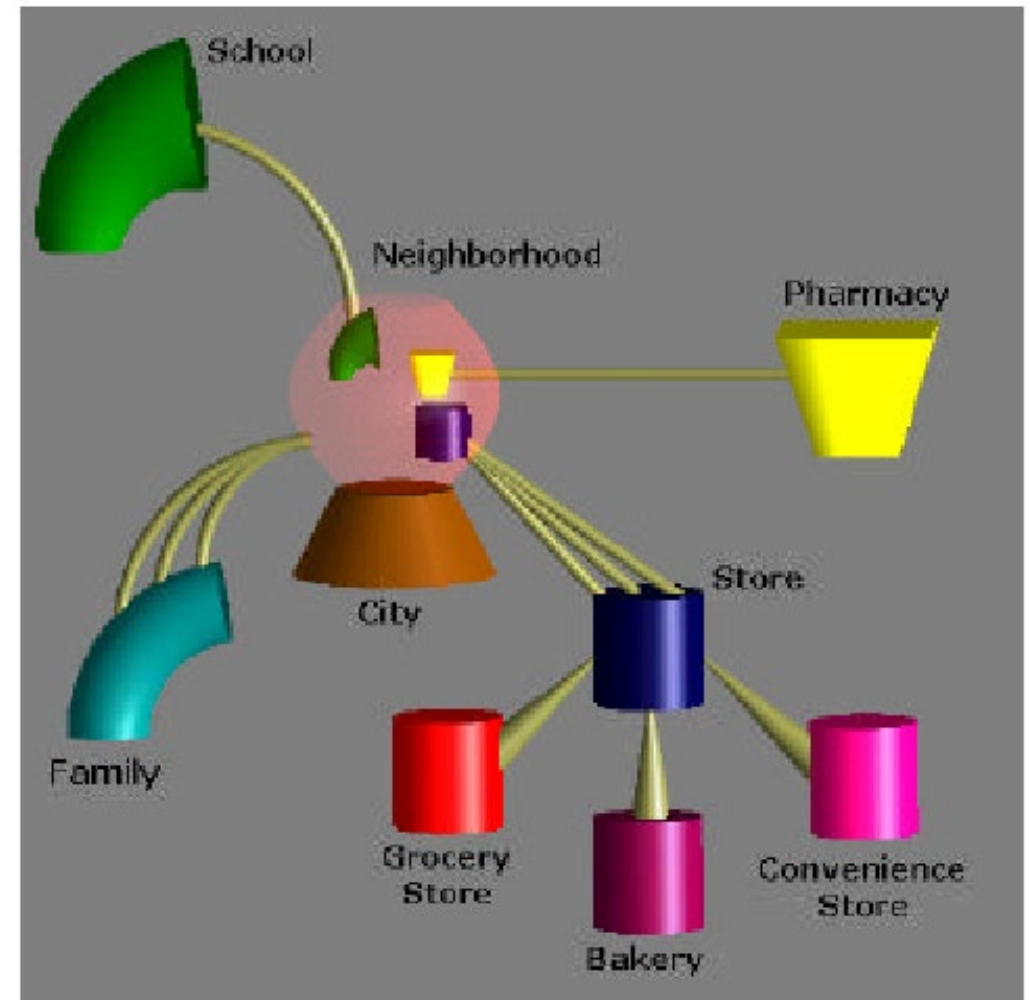
37

Approach:

Use geons to better distinguish classes

Structural Object Recognition Theory: As objects are perceived they are decomposed into a set of 3D primitives called **geons**, together with the skeleton structure connecting them.

Source: Pourang Irani and Colin Ware. 2000.
Diagrams based on structural object perception.
In Proceedings of the working conference on
Advanced visual interfaces (AVI). 2000.
<https://doi.org/10.1145/345513.345254>



Enriched Node-Link Diagrams

Software Metrics

39

A software metric is a measure of some property of a piece of software or its specifications.

[Wikipedia: Software Metric]

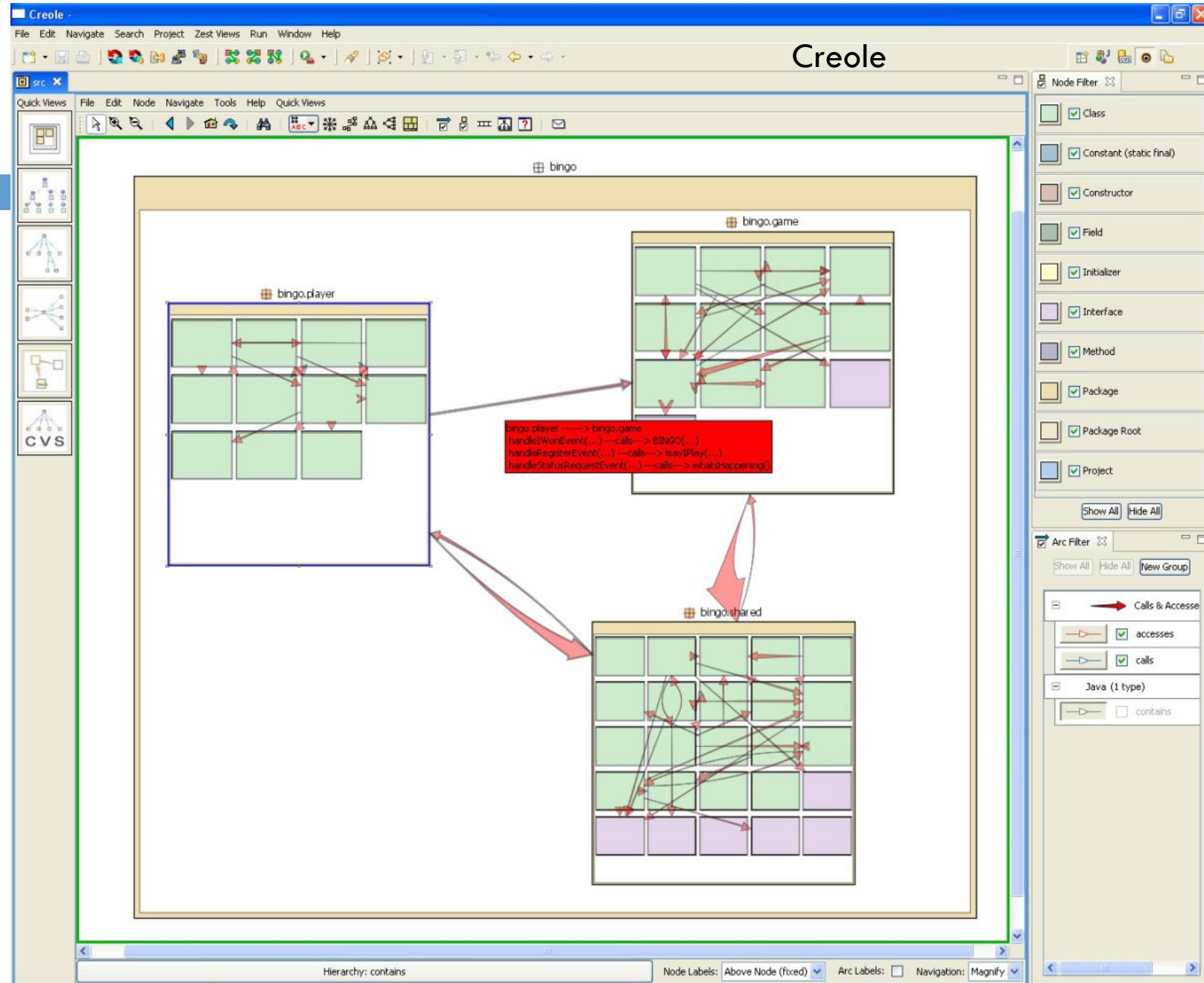
- software metrics provide additional information
 - ▣ automatic computation
 - ▣ usually: multivariate data
- may increase understanding, help to find problems

Creole/SHriMP

40

Creole is an Eclipse plug-in using SHriMP (Simple Hierarchical Multi-Perspective) nested graph visualization

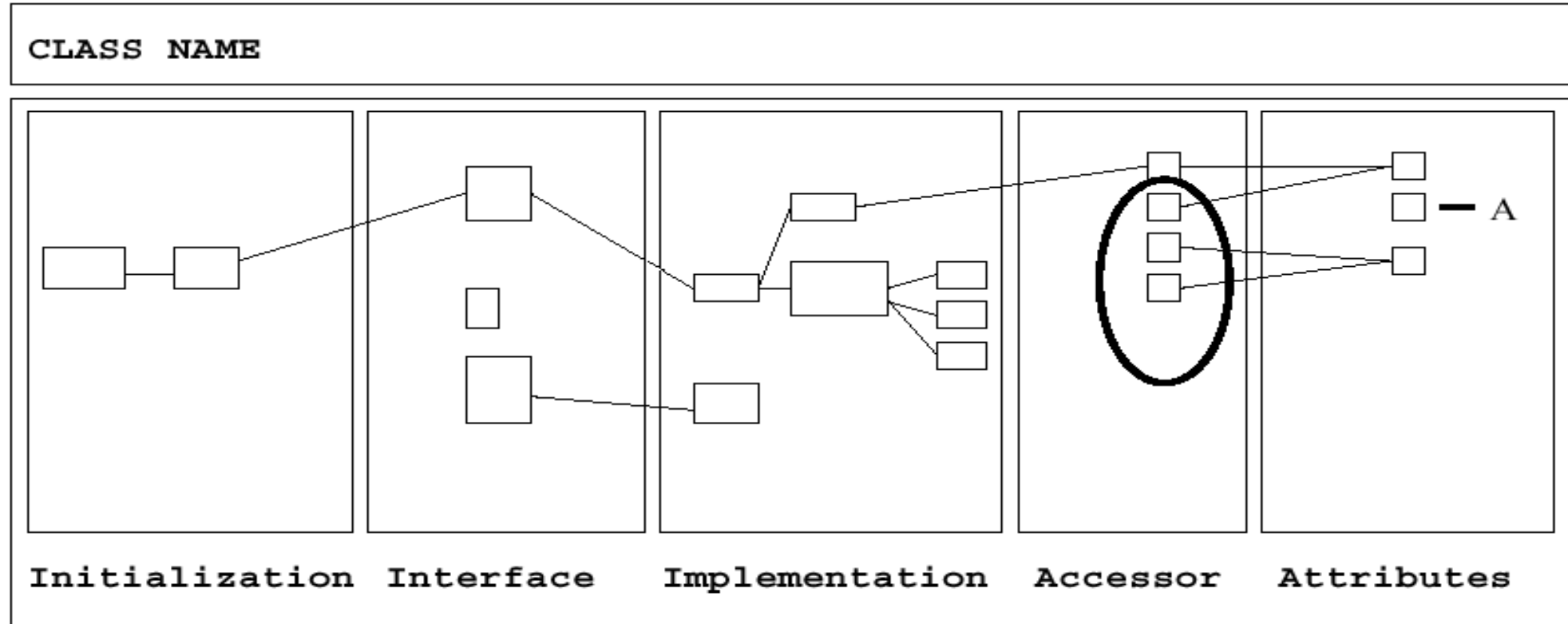
- zoomable
- nodes may show different perspectives (e.g., dependency graph, Javadoc, source code)



Class Blueprint

Lanza and Ducasse, 2001

41



- methods with substring „init“ or „initialize“
- constructors

- public or protected methods
- only invoked by init layer within the same class

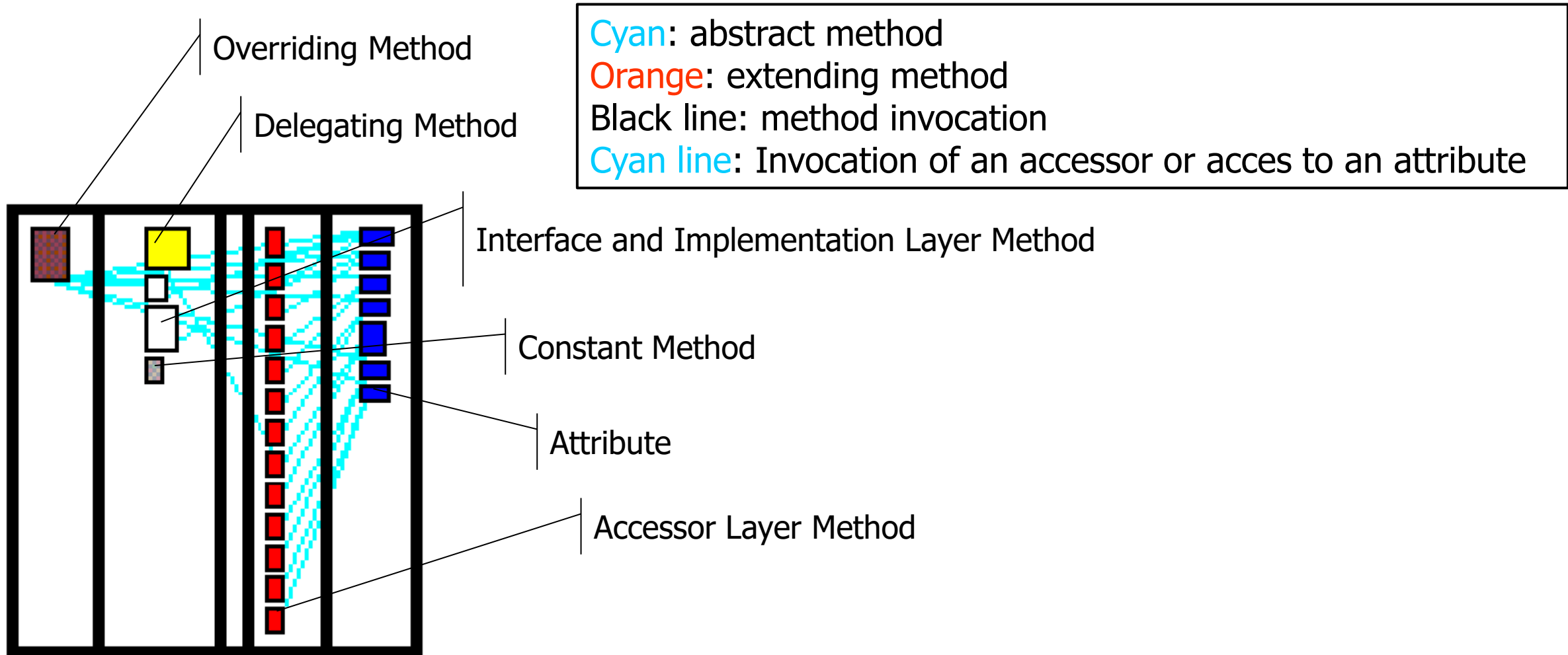
- private methods invoked by other methods in the same class

- methods to get and set the values of attributes (getter/setter)

- all attributes of the class

Class Blueprint: Colors

42

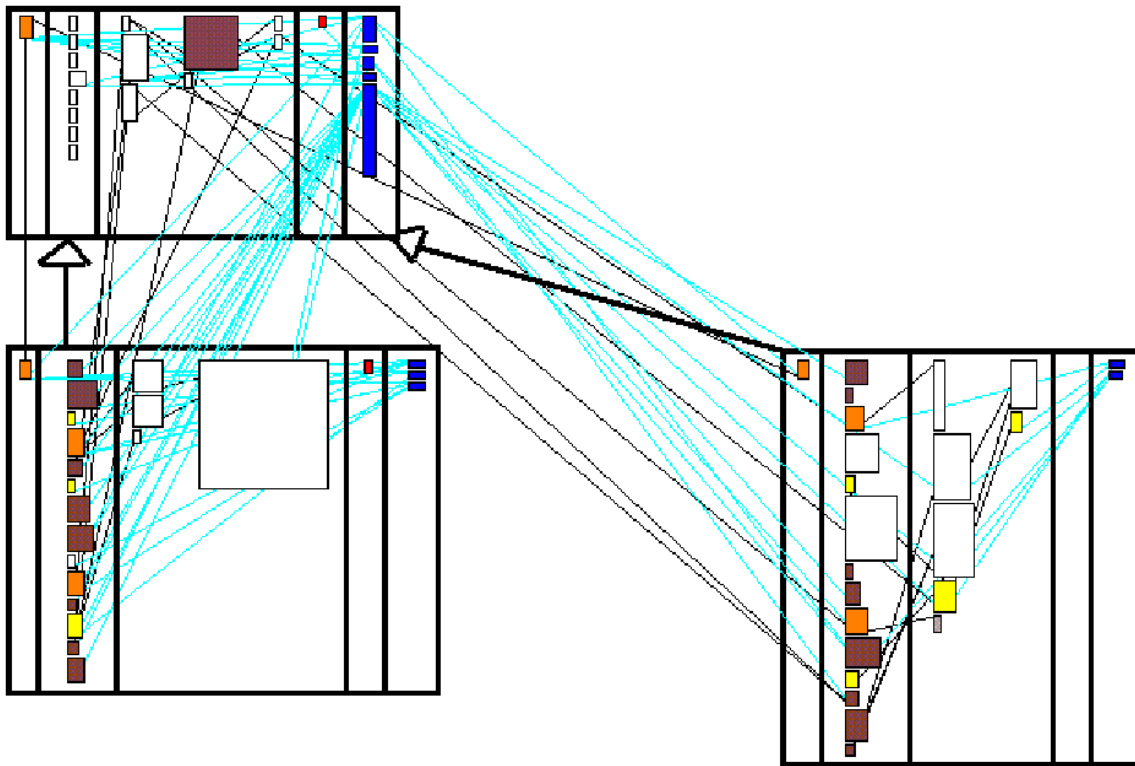


Class Blueprint: Example

43

Black line: invocation of a method

Cyan line: invocation of an accessor or access of an attribute



Based on the blueprint, classes can be categorized, e.g., as

- large implementation
- wide interface
- delegator classes

Taking inheritance into account, they can be classified as

- definers
- overriders
- extenders

Overriders and extenders can be

- talking (invoking superclass methods)
- mute (no invocation of superclass methods).

Dependency Structure Matrix

Dependency Structure Matrix (DSM)

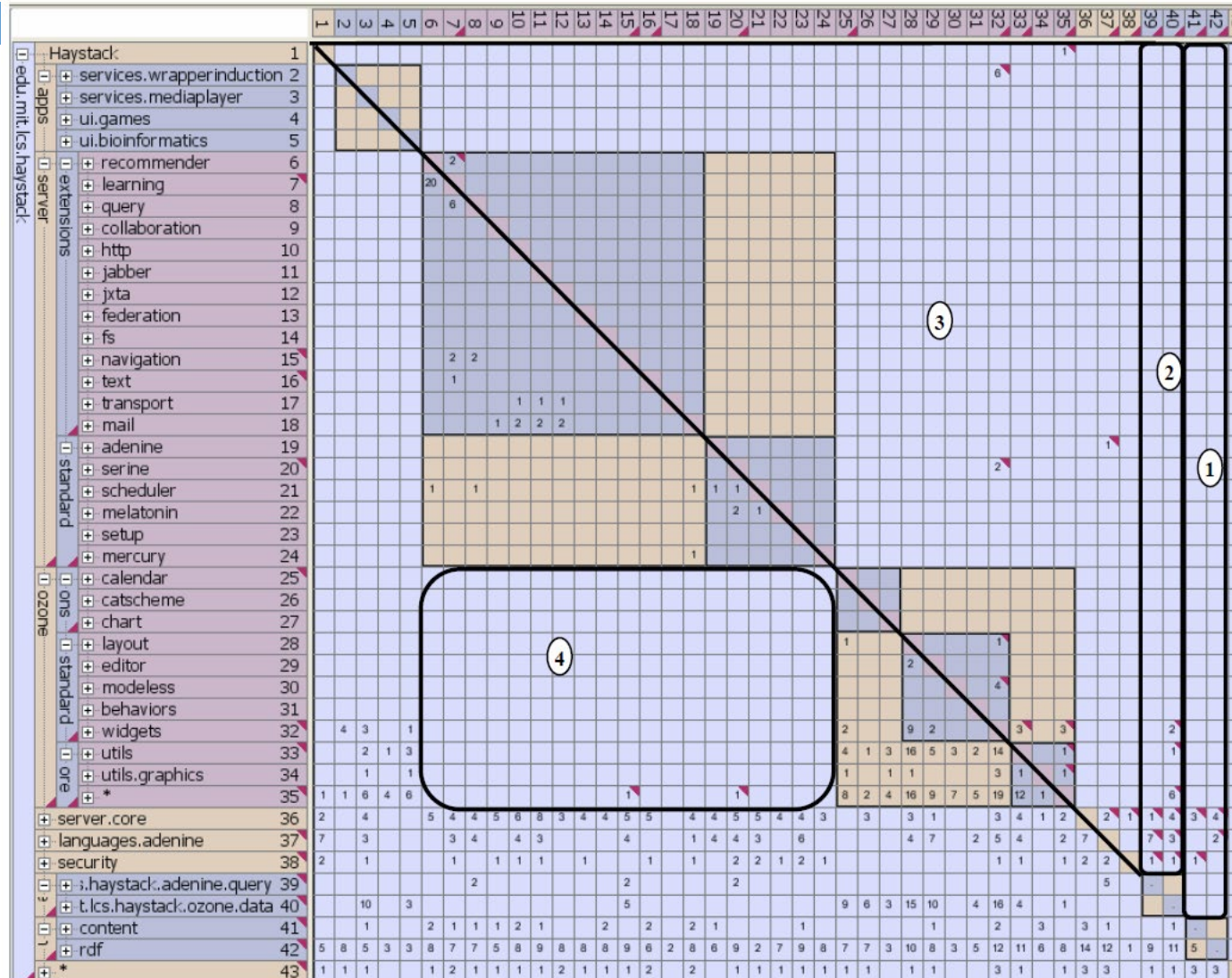
46

- also known as: Design Structure Matrix
- adjacency matrix representation of a software system
- hierarchical organization of nodes
- originally: design complex systems (engineering)
- products: Lattix, IntelliJ, sonar, structure101

			1	2	3	4	5
layer 1	Class 1	1		3	5		1
	Class 2	2					
layer 2	Class 3	3		5		2	
	Class 4	4			1		3
	Class 5	5		1		5	

Source: Neeraj Sangal, Ev Jordan, Vineet Sinha, and Daniel Jackson. 2005. **Using dependency models to manage complex software architecture**. 20th annual ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications (OOPSLA), 2005 <https://doi.org/10.1145/1094811.1094824>

47



Architecture Conformance

- layered architecture → no edges above the diagonal
- additional areas can be marked as *forbidden* (4)
- red triangles indicate design rule violations

48

Software Cities & Maps

Software Cities & Maps

49

- use 2D plane to visualize a software system
 - ▣ treemap or nested artifacts
 - ▣ dimension reduction technique
- enriched with additional information
 - ▣ software metrics
 - ▣ dependencies
- metaphors
 - ▣ city
 - ▣ map/landscape

Imsovision

50

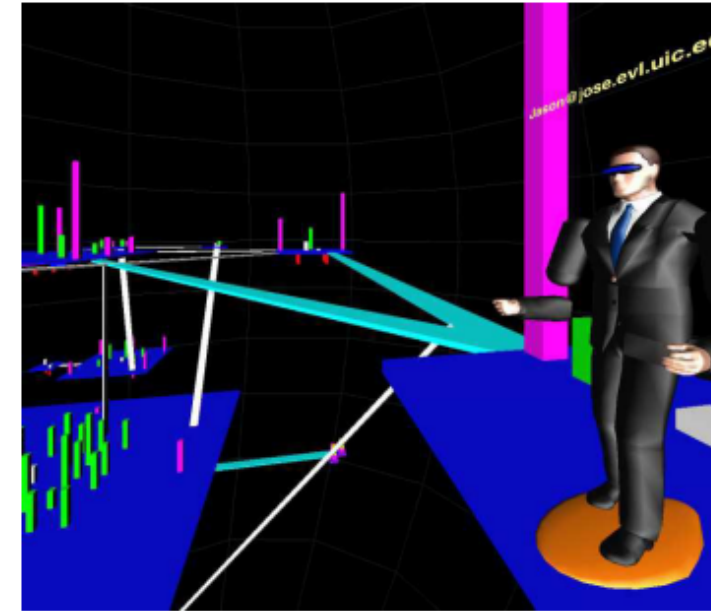
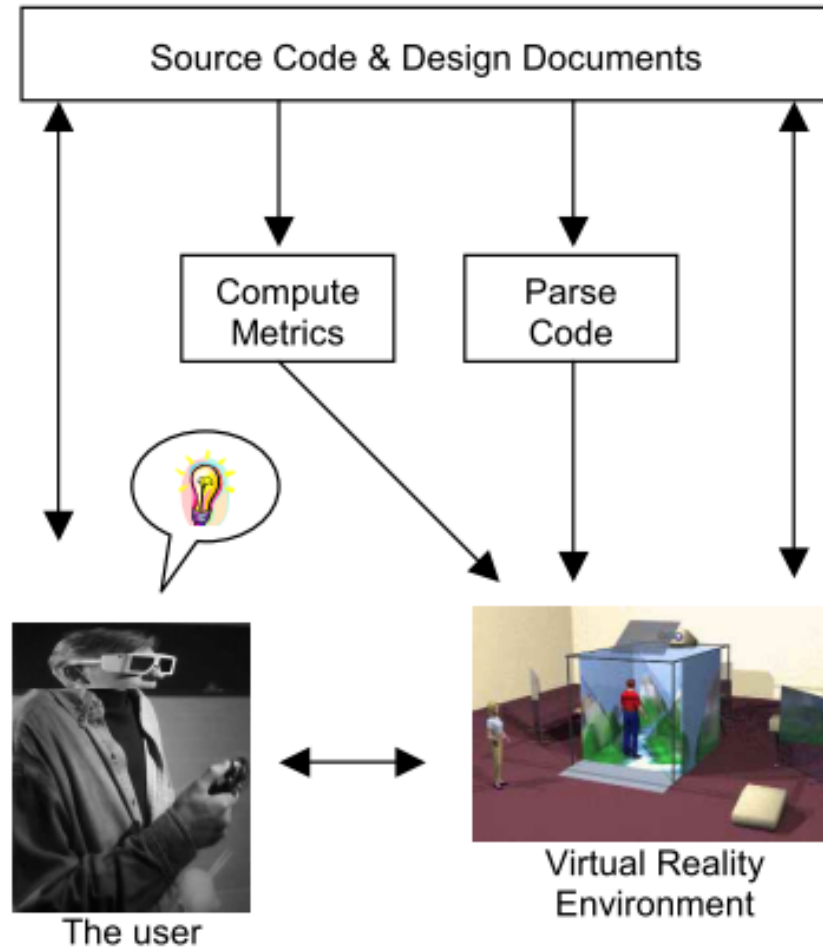


Figure 2. A remote user immersed in the VE investigating a visualization of a software system.

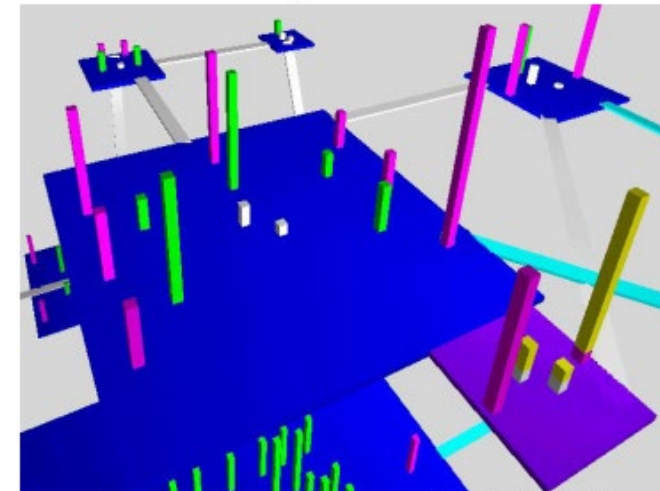


Figure 6. The Mailbox and AdminMailbox classes.

Software World

51



Figure 3 - Detail of buildings in a district; the coloured walls show whether the methods are public (lighter) or private (darker).

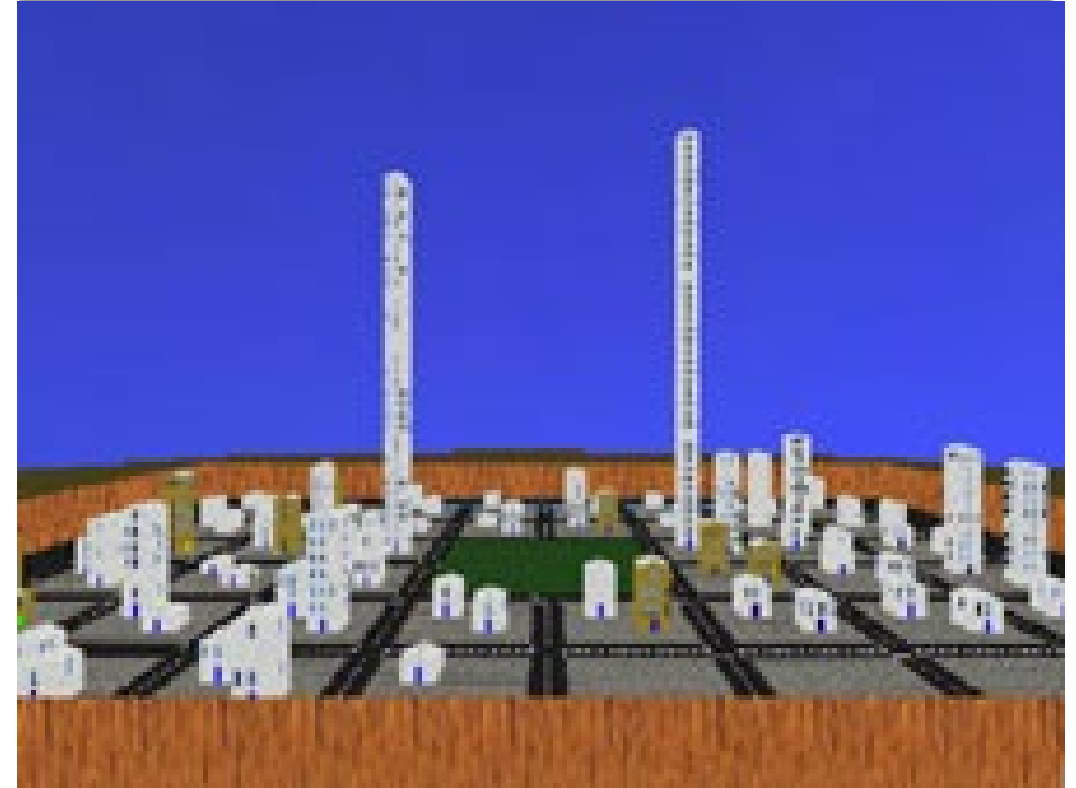
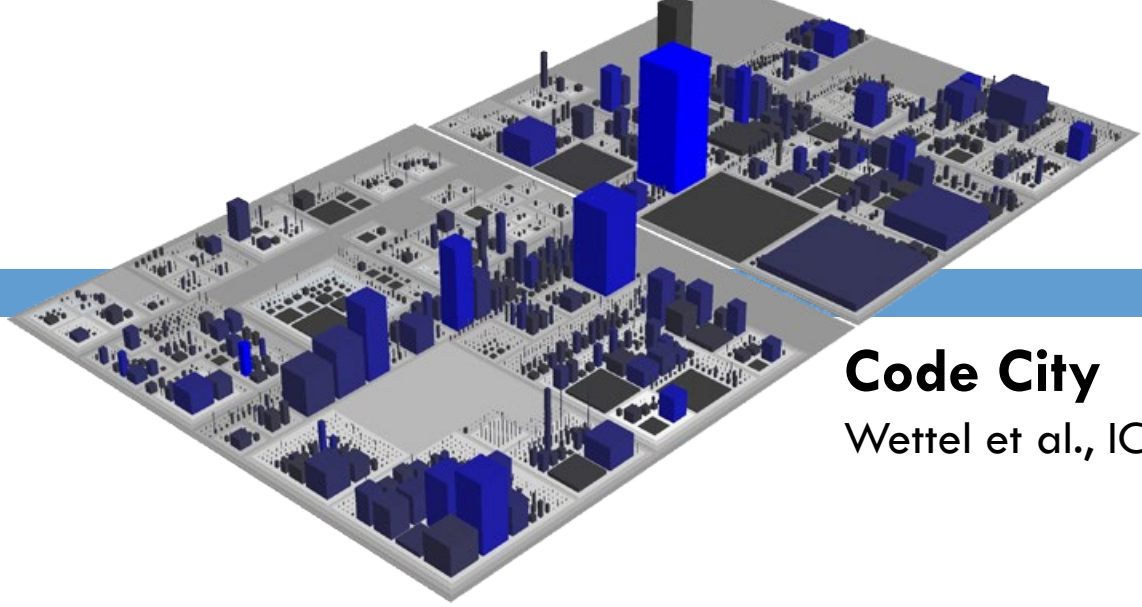


Figure 2 - Overview of a district; each district represents a class from the source code, with the methods shown as the buildings.

Dependencies

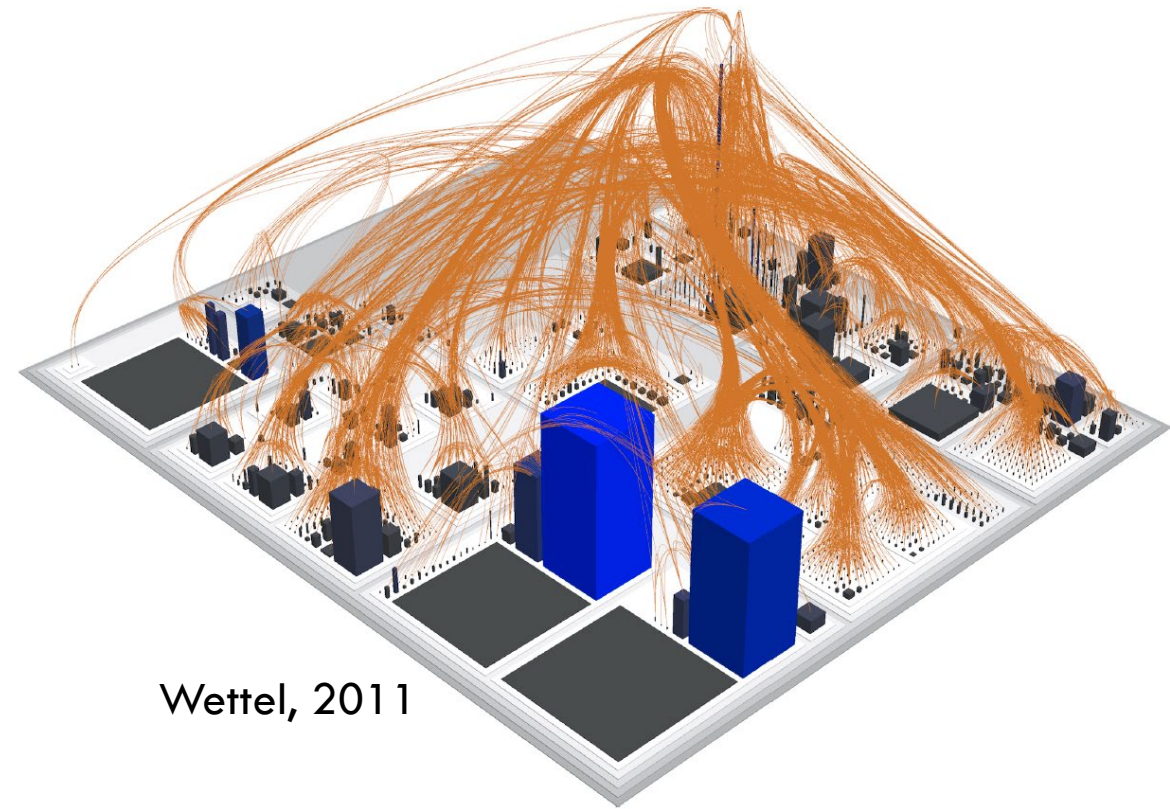
52

- bundled 3D edges show the dependency structure

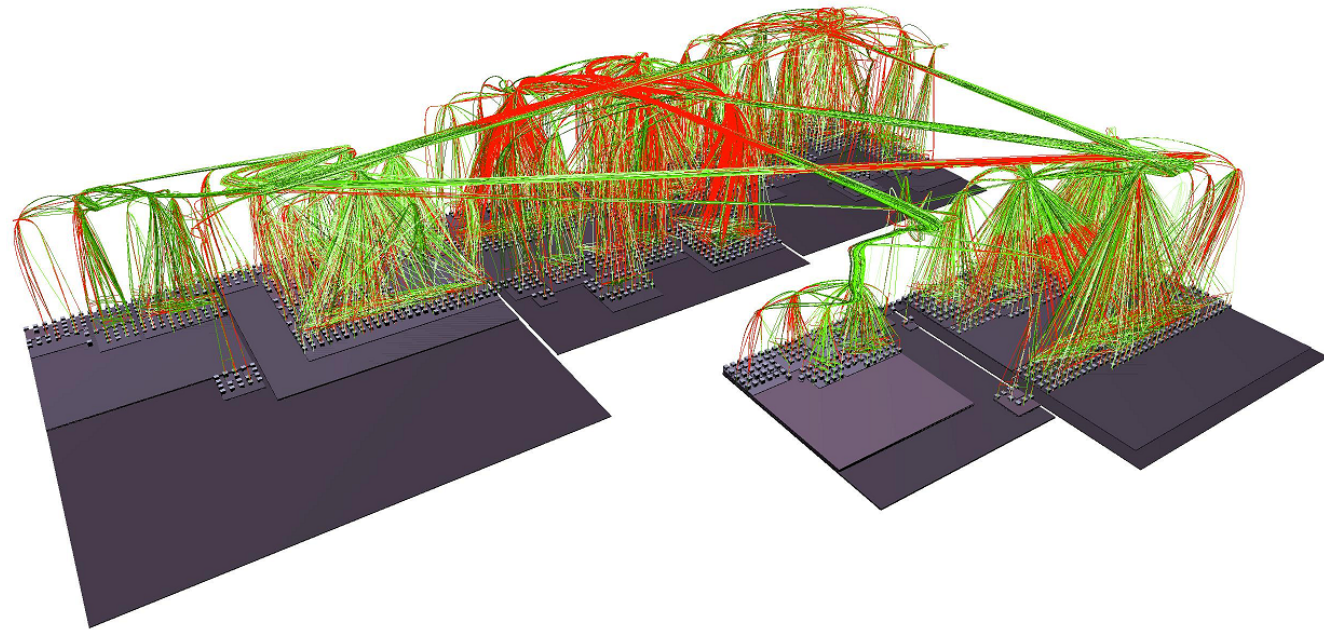


Code City

Wettel et al., ICSE 2011



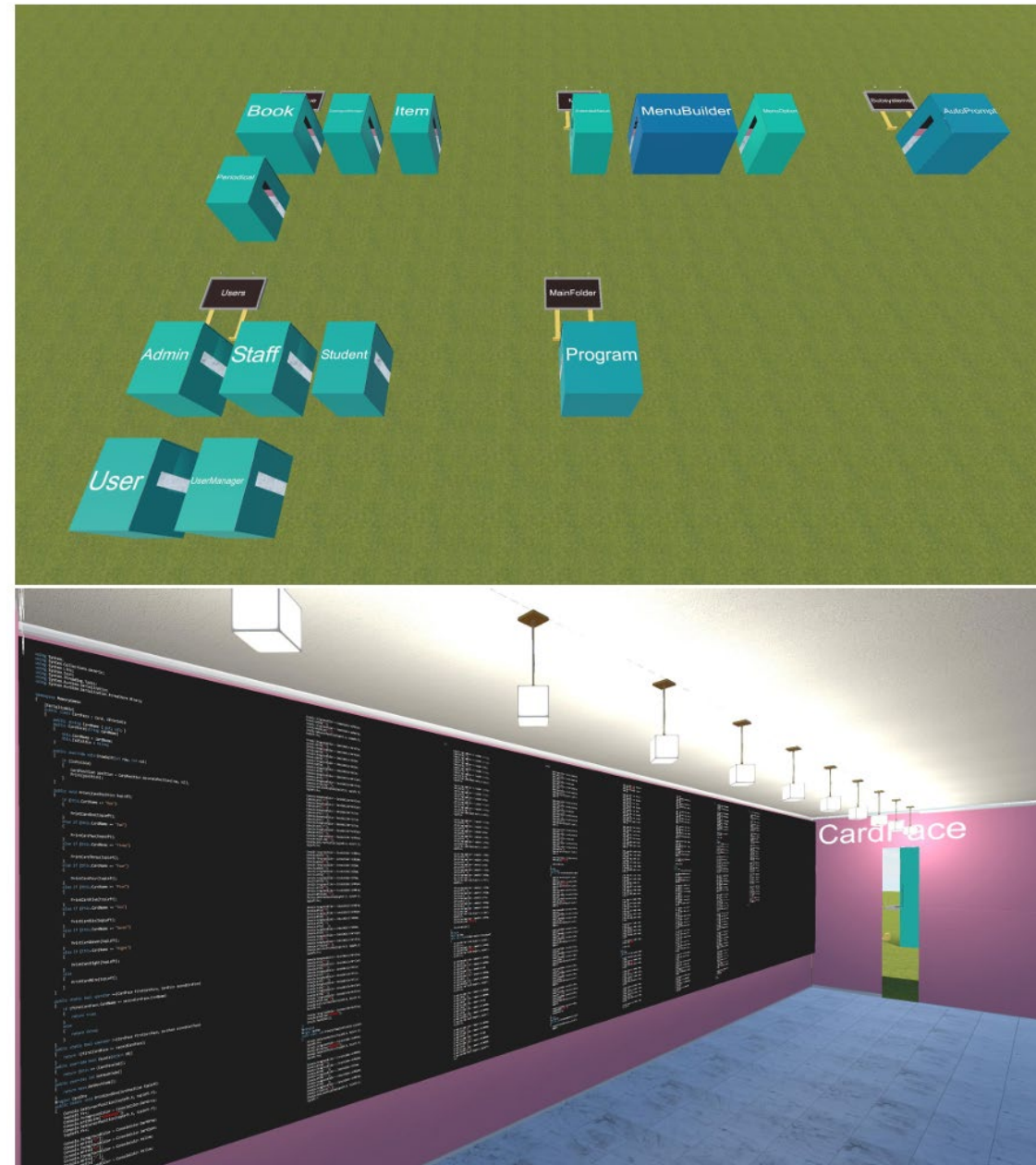
Wettel, 2011



Caserta et al., 2011

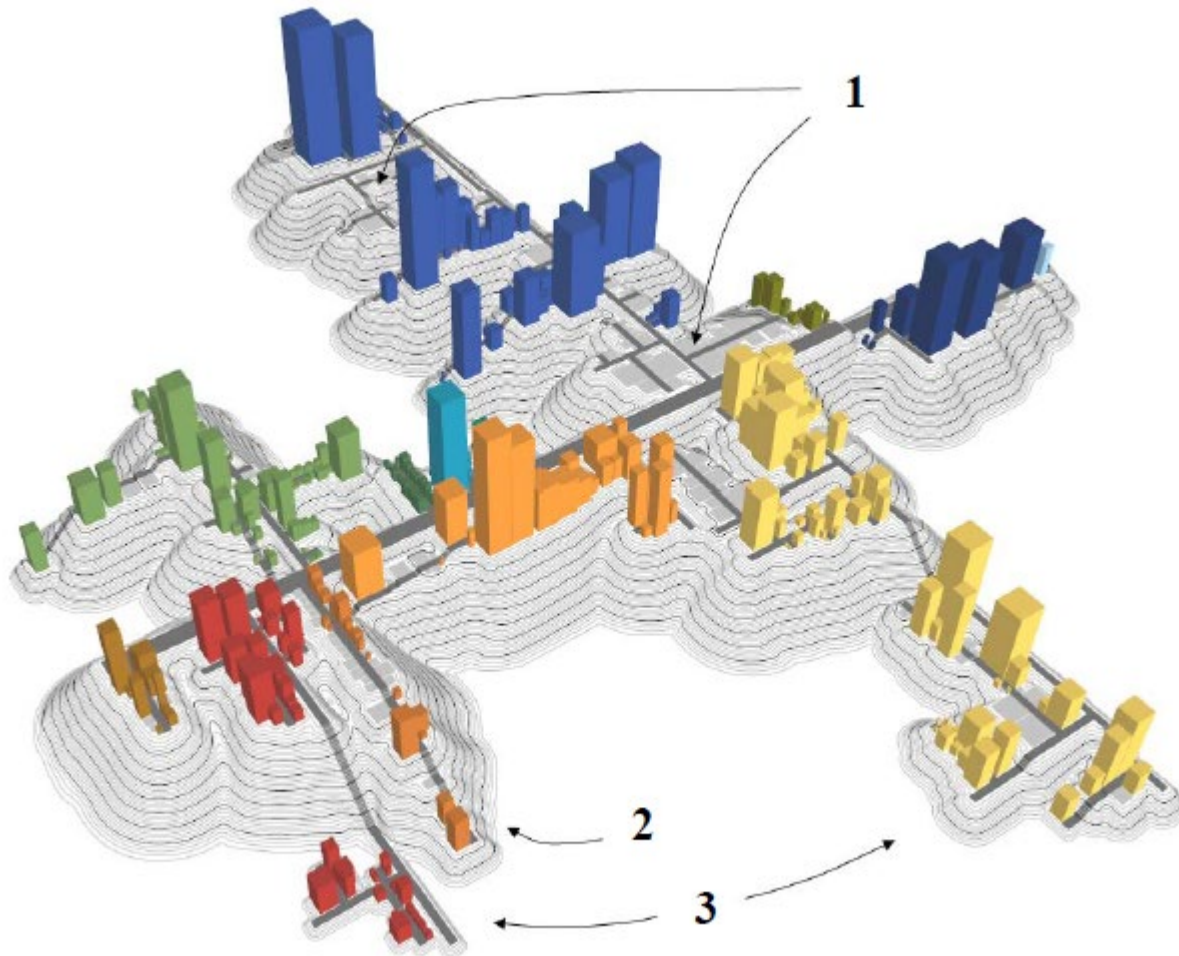
Code Park

- city metaphor more for navigation of source code
- The walls of buildings in Code Park contain source code with IDE-like syntax coloring and code navigation features.



EvoStreets

54



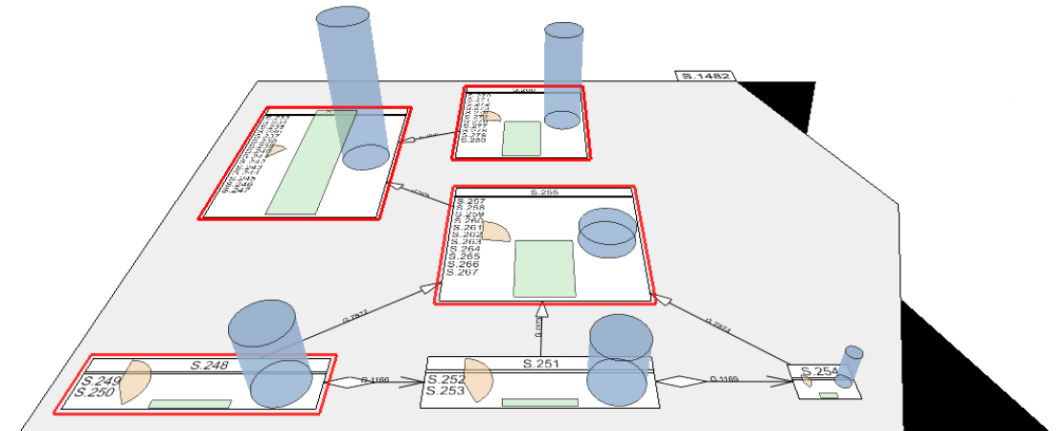
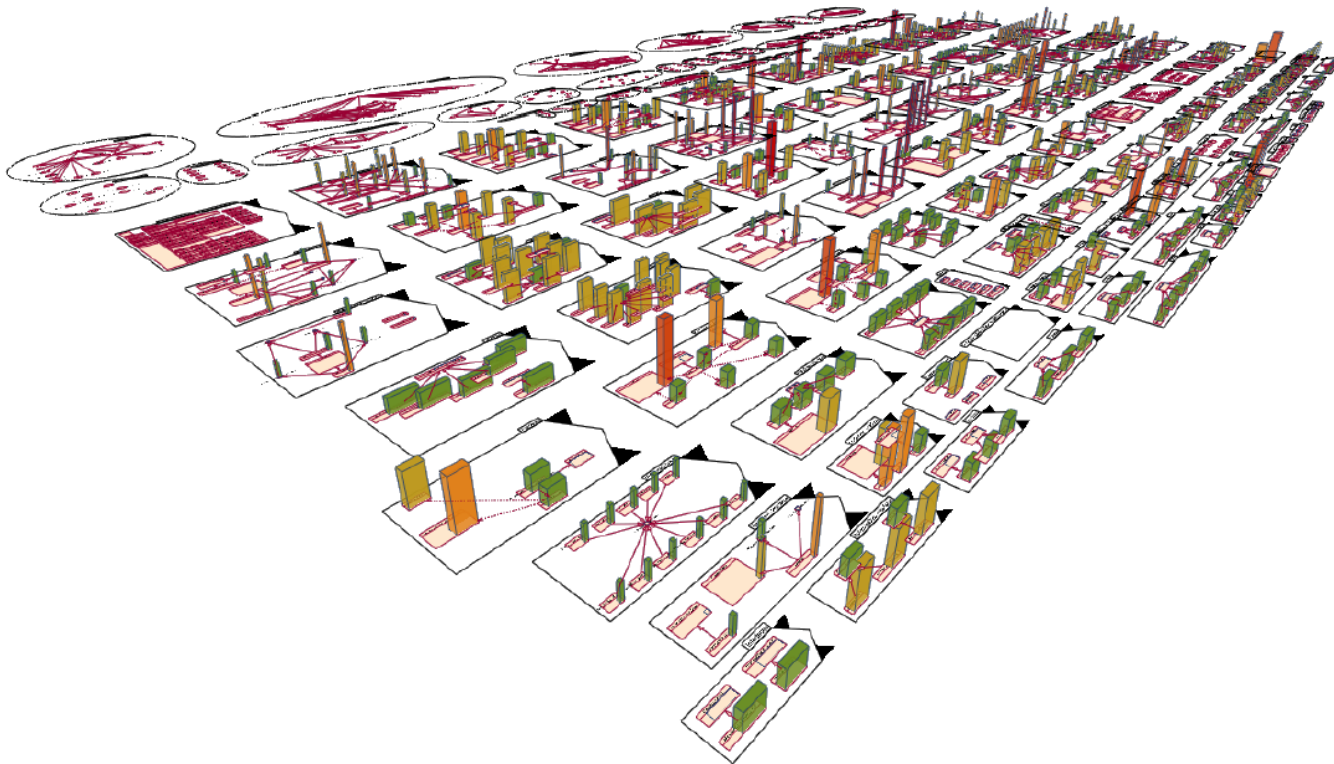
- Uses a stable layout for evolving systems
- Time is encoded the height of the hill
- Time goes from top to bottom
- Iso-lines indicate same height/age

→ Software Evolution

UML City

55

- Enriched set of UML diagrams with software metrics in 3D



Embedding in the IDE

Kuhn et al., 2010

56

The screenshot displays an IDE interface. On the left, a 'Codemap View' shows a software map with numerous nodes representing code elements. A white arrow points from a node labeled 'Grant' in the map to the corresponding code in the editor. The editor shows a Java file named 'OutsightContext.java' with the following code:

```
105  
106  
107 public boolean accepts(OutsightContext ctx) {  
108     String UID = ctx.getEnv(ctx.UID);  
109     return UID.equals(hostNode.get("LOGIN_ID"));  
110 }  
111  
112 public String toString() {  
113     return "host company";  
114 }  
115  
116 public static class EachIfShowPresentation extends Grant {  
117     private Node companyDetailsNode;  
118     public EachIfShowPresentation(Node companyNode) {  
119         companyDetailsNode = companyNode.getChild("details");  
120     }  
121     public boolean accepts(OutsightContext ctx) {  
122         return Const.TRUE.equals(companyDetailsNode.get("SHOW_PRE"));  
123     }  
124     public String toString() {  
125         return "each if show_profile";  
126     }  
127 }  
128  
129
```

At the bottom of the IDE, a 'Problems' tab is active, showing 'Members calling constructors of "Grant" - in workspace'. The list includes:

- Grant - oversight.logic.security
- EACH - oversight.logic.security.Grant
 - deny(String, Grant) : void - oversight.logic.security.SecuritySettings
 - makeAdminEntry() : Node - oversight.logic.TreeFactory
 - makeAdminRepository() : Node - oversight.logic.TreeFactory (2 matches)
 - makeCandidateEntry() : Node - oversight.logic.TreeFactory
 - makeCandidateRepository() : Node - oversight.logic.TreeFactory
 - makeCareerdayEntry() : Node - oversight.logic.TreeFactory
 - makeCompanyEntry() : Node - oversight.logic.TreeFactory (3 matches)
 - makeEvent() : Node - oversight.logic.TreeFactory (3 matches)

Software map is integrated into the IDE for navigation and for showing search results

IslandViz (AR/VR)

Source: Schreiber, Andreas, Nafeie, Lisa, Baranowski, Artur, Seipel, Peter, Misiak, Martin (2019): „Visualization of Software Architectures in Virtual Reality and Augmented Reality.“ In: *2019 IEEE Aerospace Conference*. 2019, S. 1-12, doi: 10.1109/AERO.2019.8742198

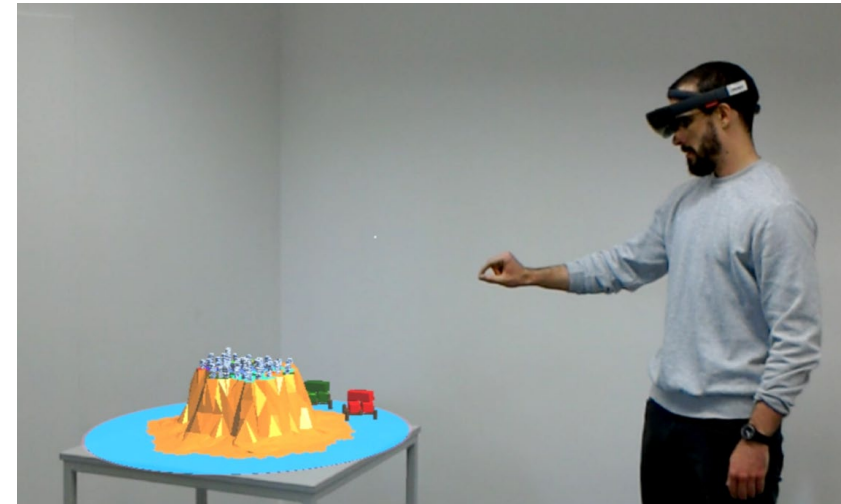
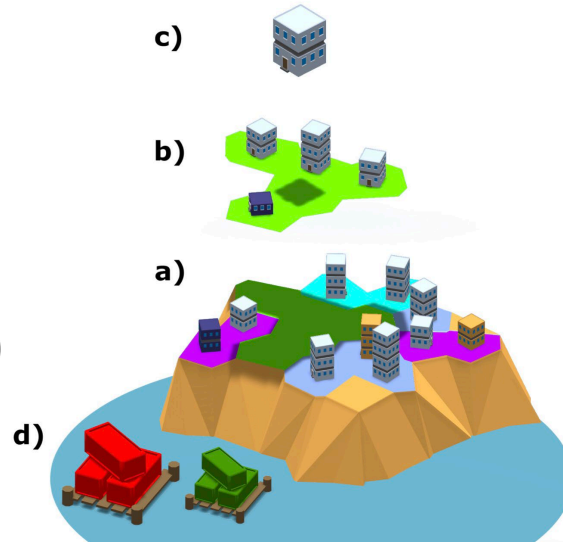
Building: Java Class

Region: Java Package

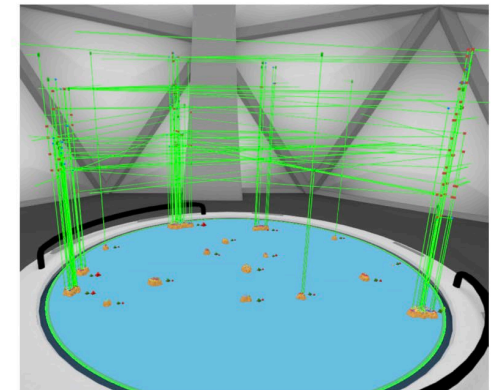
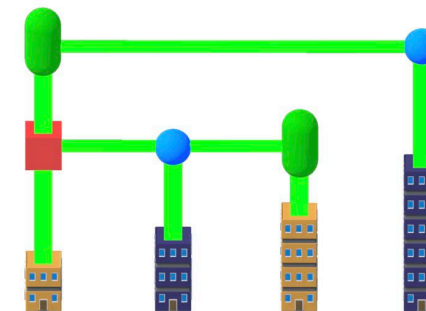
Island: OSGi Bundle
(=Component with services)

Harbor: Import and
export of packages

Layout of regions on the island



Service dependencies (OSGi)



Conclusions

Manually created architecture diagrams

- Ad-hoc diagrams
- UML diagrams

Reverse engineering enables automatic creation of

- UML diagrams
- Enriched node-link diagrams
- Dependency Structure Matrix
- Software cities

Computation of CFGs

59

Syntax of a simple programming language

$$G = \{ \begin{array}{l} S \longrightarrow \begin{array}{l} V=E \quad P \\ S;S \\ \text{if } (E) \{S\} \text{ else } \{S\} \quad P \\ \text{while } (E) \{S\} \quad P \end{array} \\ V \longrightarrow \text{Variablename} \\ E \longrightarrow \text{Expression} \\ P \longrightarrow \text{label of program point} \end{array} \}$$

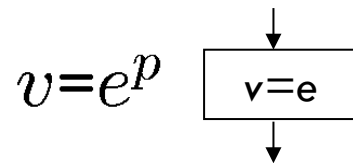
A program must not contain a program point label twice.

$$L_G(A) = \{w \mid w \text{ is a terminal word with } A \xrightarrow{G} w\}$$

Informal Computation of CFGs

60

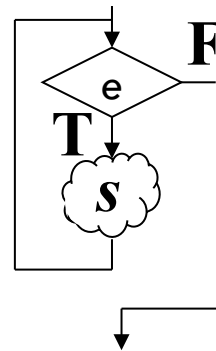
Let $s, s_i \in L_G(S)$, $v \in L_G(V)$ and $e \in L_G(E)$



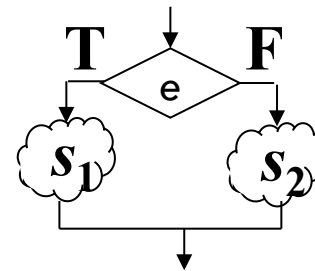
$s_1 ; s_2$



$\text{while}(e) \{s\}^p$



$\text{if}(e) \{s_1\} \text{ else } \{s_2\}^p$



Formal Computation of CFGs

61

A CFG is a tuple (V, E, in, out) where

- V is a set of nodes,
- $E \subseteq V \times L \times V$ a set of edges,
- $L = \{\epsilon, T, F\}$ a set of labels,
- and $in, out \in V$ are start and end nodes.

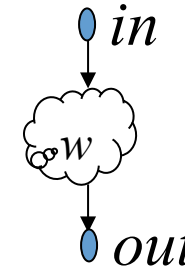
Formal Computation of CFGs

62

Let Γ be the set of all control flow graphs.
We define a function $\text{cfg} : L_G(S) \longrightarrow \Gamma$ which maps programs to control flow graphs:
 $\text{cfg}(w) = (V, E, in, out)$ where

if $w = v=e^p$ then

- $V = \{w, in, out\},$
- $E = \{(in, \epsilon, w), (w, \epsilon, out)\},$
- and in, out are two new nodes.



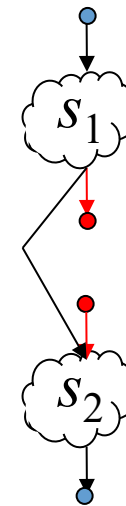
Formal Computation of CFGs

63

if $w = s_1; s_2$ then

- $V = (V_1 - \{out_1\}) \cup (V_2 - \{in_2\}),$
- $E =$

$$\begin{aligned} & (E_1 - \{(v, l, out_1) | v \in V_1\}) \\ & \cup (E_2 - \{(in_2, \epsilon, v) | v \in V_2\}) \\ & \cup \{(v_1, l_1, v_2) | (v_1, l_1, out_1) \in E_1, \\ & \quad (in_2, \epsilon, v_2) \in E_2\} \end{aligned}$$
- and $in = in_1, out = out_2.$



where $(V_1, E_1, in_1, out_1) = \text{cfg}(s_1)$
 and $(V_2, E_2, in_2, out_2) = \text{cfg}(s_2)$

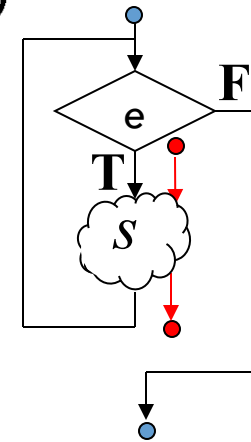
Formal Computation of CFGs

64

if $w = \text{while}(e)\{s\}^p$ then

- $V = V_0 \cup \{e^p\},$
- $E =$

$$\begin{aligned} & (E_0 - (\{(\text{in}_0, \epsilon, v) \mid v \in V_0\} \\ & \quad \cup \{(v, l, \text{out}_0) \mid v \in V_0\})) \\ & \cup \{(\text{in}_0, \epsilon, e^p), (e^p, F, \text{out}_0)\} \\ & \cup \{(e^p, T, v) \mid (\text{in}_0, \epsilon, v) \in E_0\} \\ & \cup \{(v, l, e^p) \mid (v, l, \text{out}_0) \in E_0\} \end{aligned}$$
- and $\text{in} = \text{in}_0, \text{out} = \text{out}_0.$



where $(V_0, E_0, \text{in}_0, \text{out}_0) = \text{cfg}(s)$

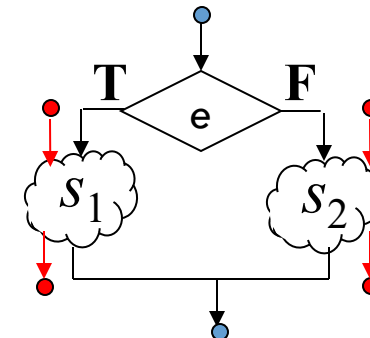
Formal Computation of CFGs

65

if $w = \text{if}(e)\{s_1\}\text{else}\{s_2\}^p$ then

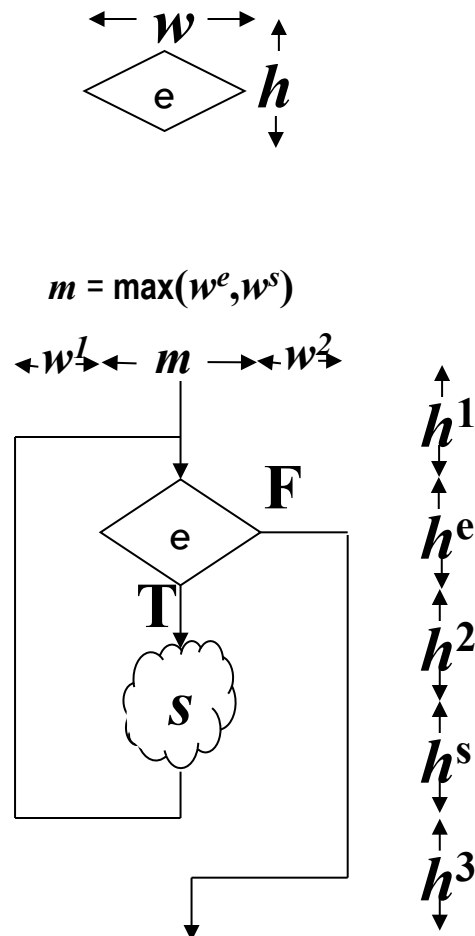
- $V = (V_1 - \{in_1, out_1\}) \cup (V_2 - \{in_2, out_2\}) \cup \{e^p\},$
- $E =$
 - $(E_1 - \{(in_1, \epsilon, v_1), (v_2, l, out_1) \mid v_1, v_2 \in V_1\})$
 - $\cup (E_2 - \{(in_2, \epsilon, v_1), (v_2, l, out_2) \mid v_1, v_2 \in V_2\})$
 - $\cup \{(in, \epsilon, e^p)\}$
 - $\cup \{(e^p, T, v) \mid (in_1, \epsilon, v) \in E_1\}$
 - $\cup \{(v, l, out) \mid (v, l, out_1) \in E_1\}$
 - $\cup \{(e^p, F, v) \mid (in_2, \epsilon, v) \in E_2\}$
 - $\cup \{(v, l, out) \mid (v, l, out_2) \in E_2\}$
- and in, out are two new nodes.

where $(V_i, E_i, in_i, out_i) = \text{cfg}(s_i)$



Template-based Layout of CFGs

66



- We use rectangular boxes and place *in* and *out* nodes in the middle of the upper and lower borders.
- The height depends on the font size and the width also on the length of the expression
- Heights h^1 , h^2 and h^3 are fixed. So we can compute $H = h^1 + h^2 + h^3$ beforehand.
- As a result we have $h = H + h^e + h^s$.
- Analogously:

$$W = w^1 + w^2 \text{ and } w = W + \max(w^e, w^s)$$

Template-based Layout of CFGs

67

We define a function $\text{box} : L_G(S) \longrightarrow \mathcal{R} \times \mathcal{R}$ which maps programs to the sizes of the box to layout its control flow graph:

$\text{box}(e) = (w, h)$ where w, h depend on the font.

$\text{box}(s_1; s_2) = (W + \max(w_1, w_2), H + h_1 + h_2)$
where $(w_i, h_i) = \text{box}(s_i)$

$\text{box}(\text{while}(e)\{s\}) = (W + \max(w_e, w_s),$
 $H + h_e + h_s)$
where $(w_e, h_e) = \text{box}(e)$ and $(w_s, h_s) = \text{box}(s)$